Qualcomm Technologies, Inc.

Camera Sensor Driver Bringup Guide

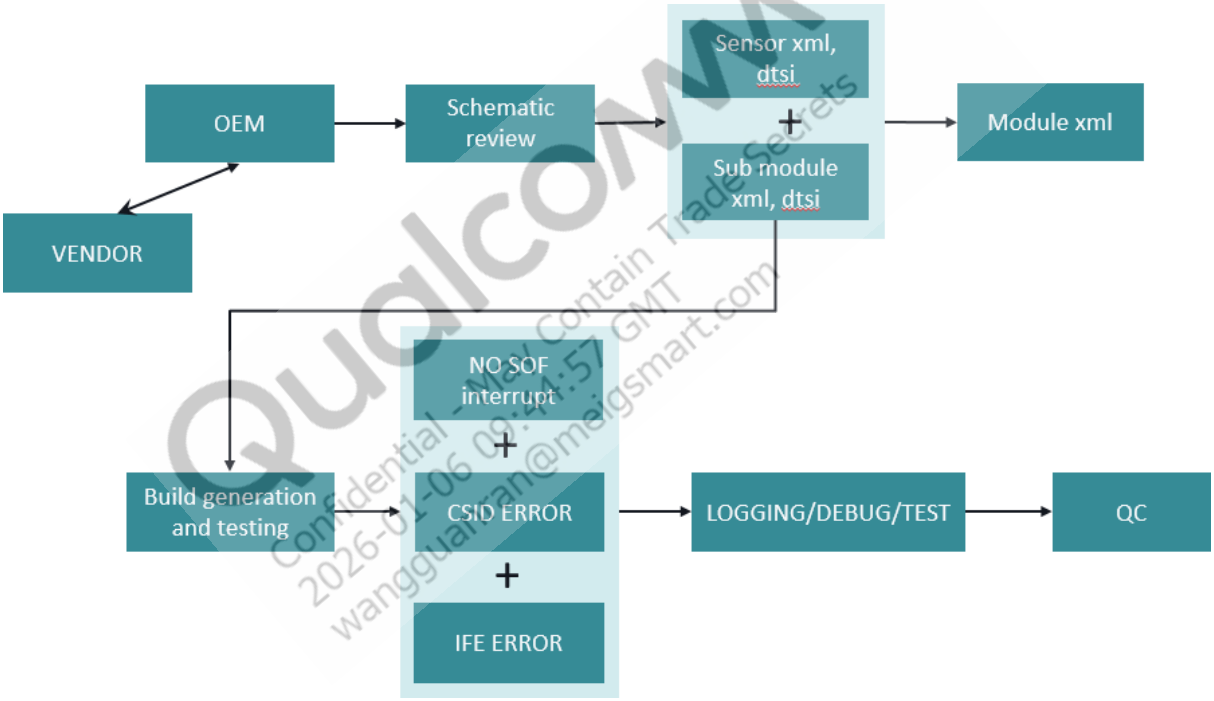
80-P9301-97 YD

July 14, 2025

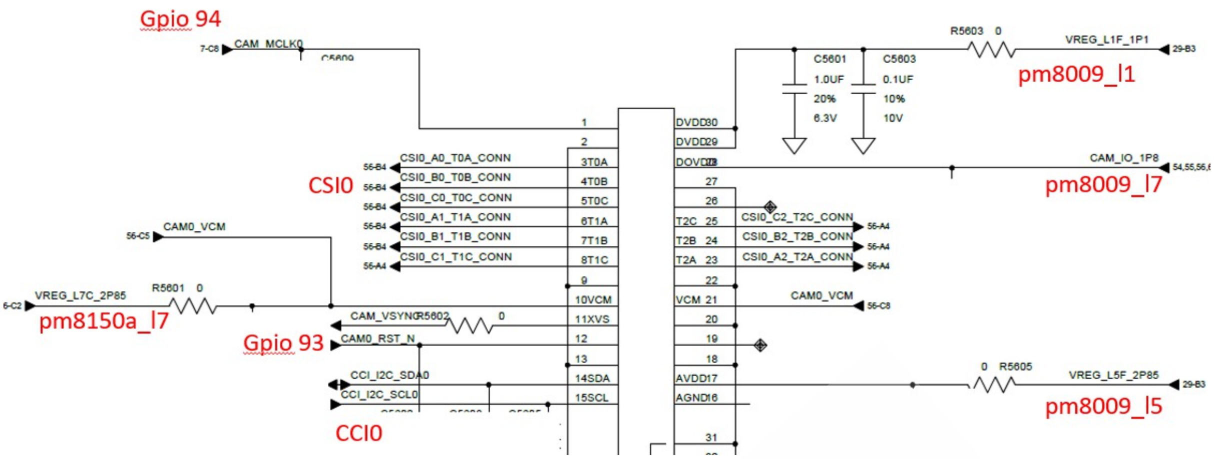
Contents

1 Overview

This document provides guidelines for driver development and describes how to bring up the camera sensor driver on the Android platform. The document also describes the terms used in the sensor driver and provides examples of the sensor configuration parameters. The following figure illustrates the workflow for sensor bring up and debug.



Before software customization, review the schematic to understand the sensor module pins connection (e.g., VDD_IO, CCI/CSI interface, etc.) as shown below. Also get the sensor power on/off timing sequence, sensor sub modules integrated (OIS, actuator, etc.). The recommendation of MCLK is 19.2Mhz. If OEM keeps seeing CSID issue for specific sensor mode, such as in higher data rate, start MIPI compliance test to rule out board level design issue.



The camera sensor driver bringup consists of the following steps:

1. Locate the driver and module configuration XML files.
2. Generate the binary files.
3. Compile the build.

File locations

The files at the following locations are modified during build compilation.

- Sensor driver XML files are at `proprietary\chi-cdk\oem\qcom\sensor\sensor_name\sensor_name_sensor.xml` (e.g., `imx586\imx586_sensor.xml` and `imx586\imx586_pdaf.xml`).
- Module configuration files are at `chi-cdk\oem\qcom\module\module_name_module.xml` (e.g., `semco_imx586_module.xml`)
- Kernel dts files are at `kernel/msm-4.19/arch/arm64/boot/dts/vendor/qcom/camera/target_name-camera-sensor-platform.dtsi`
- For SM8450 and onwards, kernel dtsi files are present in the following path: `vendor\qcom\proprietary\camera-devicetree\target_name-camera-sensor-platform.dtsi`
- Submodule driver XML files are at `chi-cdk\oem\qcom\sub-module_name\sub-module_name_sub-module.xml` (e.g., `chi-cdk\oem\qcom\actuator\ak7374_actuator.xml`).
- The driver binary in the device vendor makefile to be included in the build is at `vendor/qcom/ proprietary/common/config/device-vendor.mk` (e.g., `MM_CAMERA += com.qti.sensormodule.<sensor_name>.bin`).

1.1 Generate binary files

1. Ensure that all register settings and power on/off sequences are aligned to data sheets, for the bringup of the sensor/actuator and EEPROM are ready before driver creation.
2. As part of building the CHI-CDK code, the sensor BIN files will be automatically generated under `out\target\product\<chipset>\vendor\lib64\camera`.
3. Push the BIN files to the `/vendor/lib64/camera` folder with the appropriate root permissions. The target loads the XML files that were converted to binaries for camera software use.

2 Sensor bring-up guidelines

2.1 Sensor software configuration

Sensor-specific XML

Every sensor has an associated configuration XML file that defines features, such as power settings, resolution, initialization settings, and exposure settings.

The complete settings are present at the <sensorDriverData></sensorDriverData> node of this XML file and refer to chi-cdk/api/sensor/camxsensordriver.xsd for more detail.

The sensor's active duration should be between the FS (Frame start) – FE (Frame end) ~= 1/fps. For example, for 30 fps mode FS–FE > 30msec and in 60 fps mode the FS-FE> 14 msec. This is to avoid sensor transmitting data in burst mode.

Sensor information nodes

slaveInfo

The slaveInfo node in the XML file contains the information used by the driver while probing the sensor.

Field	Description
slaveInfo	Contains the sensor slave information and power settings.
sensorName	Name of the sensor. Example: imx230
slaveAddress	8 bit or 10 bit slave address Example: 32 For 0x20 slave address, the field value is 32. This will be used during sensor probe.
regAddrType	**Register address/data size in bytes. Example:** • 1 = Byte address • 2 = Word address • 3 = 3 byte address • 4 = Address type max
regDataType	Register address/data size in bytes. Example: * 1 = Byte data * 2 = Word data * 3 = Double word data * 4 = Data type max
sensorIdRegAddr	Register address for sensor ID. Example: 22
sensorId	Sensor ID. Example: 560
sensorIDMask	Mask for sensor ID. Sensor ID may only be a few bits. Example: 4294967295
i2cFrequencyMode	I2C frequency mode of slave. Example: FAST Supported modes are: * STANDARD (100 kHz) * FAST (400 kHz) * FAST_PLUS (1 MHz) * CUSTOM (custom frequency in DTSI)
powerUpSequence	Contains the power-up configuration sequence required to control the power to the device while turning it on. Example: <powerUpSequence> <powerSetting> <configType>RESET</configType> <configValue>0</configValue> <delayMs>0</delayMs> </powerSetting> </ powerUpSequence >
powerSetting	Contains power configuration type, value, and delay.
configType	Power configuration type. Example: MCLK Supported types are: * MCLK * VANA * VDIG * VIO * VAF * RESET * STANDBY
configValue	Power configuration type. Example: 19200000 MCLK recommended value is 19200000 (19.2Mhz)
delayMs	Delay in milliseconds. Example: 1

refAddrInfo

The register information node contains the configuration register addresses for various sensor features such as set gain, frame length lines, and test pattern generation.

Field	Description
regAddrInfo	Contains the information about the register addresses for various sensor settings.
xOutput	Controls the program width. Example: 844 If the register address is 0x34C, then the value of this field is 844.
yOutput	Controls the program height. Example: 846
frameLengthLines	Programs the frame length lines. Example: 832
lineLengthPixelClock	Programs the line length pixel clock. Example: 834
coarseIntgTimeAddr	Programs the coarse integration time. Example: 514
middleCoarseIntgTimeAddr	Programs the coarse integration time of middle exposure frame. Example: 400
shortCoarseIntgTimeAddr	Programs the coarse integration time of short exposure frame. Example: 350
globalGainAddr	Program the gain channel. Example: 516
middleGlobalGainAddr	Program the gain channel corresponding to middle exposure. Example: 14
shortGlobalGainAddr	Program the gain channel corresponding to short exposure. Example: 10
digitalGlobalGainAddr	Program the digital gain channel. Example: 3
middleDigitalGlobalGainAddr	Program the digital gain channel corresponding to middle exposure. Example: 2
shortDigitalGlobalGainAddr	Program the digital gain channel corresponding to short exposure. Example: 1
digitalGainRedAddr	Programs digital gain for the red channel. This address is optional but is required if supported by the sensor. Example: 528
digitalGainGreenRedAddr	Programs digital gain for the green-red channel. This address is optional but is required if supported by the sensor. Example: 526
digitalGainBlueAddr	Register address to program digital gain for the blue channel. This address is optional but is required if supported by the sensor. Example: 530
digitalGainGreenBlueAddr	Programs digital gain for the green-blue channel. The address is optional but is required when supported by the sensor. Example: 532
testPatternRAddr	Programs manual test pattern value for the red channel. Example: 1538
testPatternGRAddr	Programs manual test pattern value for the red channel. Example: 1540
testPatternBAddr	Programs manual test pattern value for the red channel. Example: 1542
testPatternGBAddr	Programs manual test pattern value for the red channel. Example: 1544

resolutionInfo

The resolution information node contains the resolution settings and configuration.

Node	Description
`resolutionInfo`	Specifies the configuration and settings for all the resolutions.
`resolutionData`	Specifies the configuration data for one resolution. The number of sensor supported resolutions is equal to the number of `resolutionData` nodes. First node of the `resolutionData` should always point to the full resolution configuration of the sensor.
`colorFilterArrangement`	Specifies the color filter arrangement of the sensor. For example, `BAYER_RGGB`. Supported filter arrangements are: • `BAYER_BGGR` • `BAYER_GBRG` • `BAYER_GRBG` • `BAYER_RGGBn` • `BAYER_Y` • `YUV_UYVY` • `YUV_YUYV`
`numPixelsPerColor`	Specifies the number of pixels per color in the color filter arrangement. Default is 1. Set this to 4 for QCFA modes.
`streamInfo`	Information related to the stream configuration. For example: <streamInfo> <streamConfiguration> <vc range="[0,3]">0</vc> <dt>43</dt> <frameDimension> <xStart>0</xStart> <yStart>0</yStart> <width>4608</width> <height>2592</height> </frameDimension> <bitWidth>10</bitWidth> <type>IMAGE</type> </streamConfiguration> </streamInfo> The `frameDimension` in each resolution should not be larger than `activeDimension`.
`streamConfiguration`	Information related to the stream data.

Node	Description
vc	Specifies the virtual channel of the data. Valid values for virtual channel range is from 0 to 31. Check sensor specific documentation for details. A maximum of two virtual channels per stream can be defined, one even and one odd. These must be defined for modes that support seamless mode switch. If supporting multi-VC for seamless mode switch as well as SHDR/QCFA use case, add even and odd VC value.
vcMAP (XR only)	Contains information about the VC mapping between sensor VC and expected output VC to host processor.
dt	Specifies the DT of the stream. For example: 43. Default value is 0x2B (10 bit RAW)
channelID (XR only)	Contains detail of Channel ID, which is same as virtual channel
trackerID (XR only)	Contains detail of Tracker ID
frameDimension	Specifies the frame dimension using x and y start coordinates, and the total width and height of the image.
xStart	Specifies the X coordinate of start of the image.
yStart	Specifies the Y coordinate of start of the image.
width	Specifies the width of the image.
height	Specifies the height of the image.
bitWidth	specifies the bit width of the data.
type	Specifies the stream type. For example: IMAGE. Supported stream types are: • BLOB • IMAGE: Long exposure frame when there are multiple image frames generated, and default image frame when there is a single image frame. • IMAGE_SHORT: Short exposure frame when there are multiple image frames generated. • PDAF: Phase Difference Auto Focus. Phase Difference (PD) is based on PDAF data generated by the sensor during same frame blanking period as the image. • HDR: High Dynamic Range. Any histogram stat data generated by the sensor during the same frame blanking period as the image. • META: Sensor frame meta stream output.
lineLengthPixelClock	Specifies the line length pixel clock of the frame. Typically, this value is the active width + blanking width.
frameLengthLines	Specifies the frame length lines of the frame. Typically, this value is the active height + blanking height.
minHorizontalBlanking	Specifies the minimum horizontal blanking interval in pixels.
minVerticalBlanking	Specifies the minimum horizontal blanking interval in lines.

Node	Description
`outputPixelClock`	This value should be obtained from the sensor vendor for a given mode of operation. Incorrect setting could cause PHY/CSID errors, incorrect IFE resource/clock selection behavior, or higher than required power.
`horizontalBinning`	Horizontal binning value.
`verticalBinning`	Vertical binning value.
`framerate`	Maximum frame rate.
`laneCount`	Specifies the number of data lanes on which the sensor outputs data for a given mode of operation. The maximum data lane capability (available in the datasheet) of the sensor along with the sensor register settings configured in the driver determines the value.

Node	**Description**
`settleTimeNs`	Specifies the settle time in nanoseconds. The value is configured, based on the output characteristics of the sensor to ensure that the PHY transmitter of the sensor does not have sync issues with the PHY receiver of chipsets. • DPHY: `settleTimeNs= ((85ns + 6UI)/T(Timer_clk) ) - 10`. Where, `TIMER_CLK` refers to the operating frequency of PHY interface to which the camera sensor is connected (for example, `CAMSS_PHY0_CSI0PHYTIMER_CLK` for `PHY0`). This value is set in the camera..dtsi file. Most of the time this value is 300Mhz but review the clock log. The `TIMER_CLK` value is 400Mhz. `T(TIMER_CLK)` is the duration of a clock cycle when the operating frequency is equal to `TIMER_CLK`, and is represented in nanosecond units. For example: – UI: Unit Interval is clock cycle time based on data rate per lane. If the sensor outputs at the rate of 1000Mbps for each data lane, UI is 1ns. – TIMER_CLK= 300Mhz -> T(Timer_clk)=3.3ns – The min value in xml is ((85+6)/3.3ns))-10=17 For SM8450 the timer clk value is 400Mhz -> T(Timer_clk)=2.5ns. The min value in xml is ((85+6)/ 2.5ns)-10=26. Refer to MIPI D-PHY v1.2 spec for more details. • C-PHY: Ideal `settleTimeNs = {t3-prepare+[t3-preamble]/3}/Ttimer_clock -10` Work with the sensor vendor to set t3-prepare to the lowest value possible and refer to MIPI C-PHY v1.2/v2.0 spec. For sensors supporting C-PHY v1.2/v2.0, ensure T3-CALPREAMBLE, T3- ASID, T3-CALALTSEQ timing are compliant to specifications as described in section 6.12.1.2 Calibration pattern format 2. – `Ttimer_clock` = PHY timer clk cycle time. Use the correct value in case customized by customer based on sensor config and output data rate. Customers are requested to share details of such customization with Qualcomm for review. Unit: nanoseconds. – `t3-prepare`: Confirm with the sensor vendor. If not sure, must be measured via logic analyzer. Can be different for each resolution mode setting. Unit: nanoseconds. – `t3-preamble`: Confirm with the sensor vendor. Can be different for each resolution mode setting. Unit: nanoseconds. Confirm with the sensor vendor that the minimum requirements from the receiver are met by the sensor. Note that the UI here is calculated as the cycle time for

Node	**Description**
`is3Phase`	Flag to know if the sensor is a three-phase sensor (C-PHY) or one-phase sensor (D-PHY).
`resSettings`	Sequence of register settings to configure the device with this resolution. Do not add stream on settings for the sensor in the initialization settings, as this enables sensor streaming before the real software stream is on and may cause PHY to miss LP sequence. For example: <resSettings> <regSetting> <registerAddr>0x114</registerAddr> <registerData>0x3</registerData> <regAddrType range="[1,4]">2</ regAddrType> <regDataType range="[1,4]">1</ regDataType> <operation>WRITE</operation> <delayUs>0x0</delayUs> </regSetting> <resSettings>
`regSetting`	Holds one register configuration and forms a unit of large-resolution register settings sequence.
`registerAddr`	Register address that is accessed.
`registerData`	If the operation is `WRITE`, `registerData` is the data value to be written into the specified register address. If the operation is `READ`, `registerData` is the number of bytes to be read from the specified register address.
`regAddrType`	Type of register address.
`regDataType`	Type of register data.
`operation`	Type of operation. Supported values are: • WRITE • READ • POLL
`delayUs`	Delay in microseconds. If not explicitly specified, the delay is zero.

Node	Description

`cropInfo`	Crop information of the frame. For example:
	`<cropInfo>`
	`  <left>0</left>`
	`  <right>0</right>`
	`  <top>0</top>`
	`  <bottom>0</bottom>`
	`</cropInfo>`

`left`	Left crop pixel information.
`right`	Right crop pixel information.
`top`	Top crop pixel information.
`bottom`	Bottom crop pixel information.
`exposureInfo`	Resolution specific exposure control information. Update if it is different from the common resolution exposure control information. If this information is not available, then info from common exposure control will be used.
`exposureType`	Information about the exposure type for which the exposure control information is being provided. Supported values are: • `DEFAULT`: Corresponds to the long exposure when there are multiple exposures and default exposure in case of single exposure mode. • `SHORT`: Corresponds to the short exposure when there are multiple exposures. • `MIDDLE`: Corresponds to the middle exposure when there are multiple exposures.
`coarseIntgTimeAddr`	Register address to program the coarse integration time for this exposure type in this resolution.
`globalGainAddr`	Register address to program the gain channel for this exposure type in this resolution.
`digitalGlobalGainAddr`	Register address to program the digital gain channel for this exposure type in this resolution.
`maxAnalogGain`	Maximum analog gain supported by the sensor for this resolution.
`minAnalogGain`	Minimum analog gain supported by the sensor for this resolution.
`maxDigitalGain`	Maximum digital gain supported by the sensor for this resolution.
`minDigitalGain`	Minimum digital gain supported by the sensor for this resolution.
`maxLineCount`	Maximum Line Count supported by the sensor for this resolution.
`minLineCount`	Minimum Line Count supported by the sensor for this resolution.

Node	Description
`verticalOffset`	Minimum offset to be maintained between the line count and frame length lines for this resolution.
`frameOffsetInfo`	Holds addresses of various frame offset registers
`middleFrameOffsetRegister`	Register address to program the frame offset value of middle exposure frame.
`shortFrameOffsetRegister`	Register address to program the frame offset value of short exposure frame.
`HDRExposureType`	Information about the HDR exposure type of this resolution. This value must be filled only if the resolution is a type of HDR. Supported values are: • `ONEEXPOSURE` • `TWOEXPOSURE`: Indicates that two exposures are used in the HDR mode (Long and Short exposure) • `THREEEXPOSURE`: Indicates that three exposures are used in the HDR mode (Long, Middle, and Short exposure)
`ZZHDRInfo`	ZZHDR color pattern and first exposure information. Supported values are: • `ZZHDRPattern`: This value represents the zzHDR pattern such as `P0P1P0P0` • `ZZHDRFirstExposure`: This value represents whether short exposure or long exposure field comes first such as `SHORTEXPOSURE`

Node	Description	
`HDR3ExposureInfo`	Information of in-sensor HDR 3 exposure. • `HDR3ExposureType`: Exposure type for 3HDR non-seamless mode • `numberOfLTCRatioRegCount`: Number of LTC ratio registers • `sensorLTCRatioAddr`: Register address to program LTC ratio • `InSensorHDR3ExpLineLengthPixelClock`: `LineLengthPixelClock` for seamless in-sensor HDR 3 exp mode switching • `InSensorHDR3ExpFrameLengthLines`: `FrameLengthLines` for seamless in-sensor HDR 3 exp mode switching • `InSensorHDR3ExpMaxAnalogGain`: `MaxAnalogGain` for seamless in-sensor HDR 3 exp mode switching • `InSensorHDR3ExpStartSettings`: `StartSetting` sequence for seamless in-sensor HDR 3 exp mode switching • `InSensorHDR3ExpStopSettings`: `StopSetting` sequence for seamless in-sensor HDR3 exp mode switching	
`RemosaicTypeInfo`	Specifies the remosaic type. Supported features are: • `SWRemosaic` • `HWRemosaic`	• `NoRemosaic`

Node	Description
`capability`	List of features/capabilities supported by the sensor. Supported features are: • AONFD • AONFDMIPI • AONMD • DEPTH • FASTAEC • FS • HFR • IHDR • INSENSORZOOM • INTERNAL • NORMAL • PDAF • QHDR • QUADCFA • SHDR • ZZHDR
`ADCReadoutTime`	Analog-to-digital conversion time for the sensor. Time specified in milliseconds. For example: 2.
`mipiFlags`	Any extra mipi parameters that receiver should take care of . Supported features are: • EPDEnabled • DSKEWCalibEnabled • PN9PatternEnabled
`maxAnalogGain`	Maximum analog again supported by current sensor mode
`QHDRInfo`	Specifies `QCFAHDRInformation`. • QHDRType: – `PIXELINTERLEAVED`: QCFA format like native pixel arrangement – `ROWINTERLEAVED`: Each exposure on different line via same VC-DT – `OWINLINEINTERLEAVED`: Each exposure on different line via different VC-DT • QHDRPattern: Valid patterns are MLSM, SMML, MSLM, LMMS, MLMS, SMLM, MSML, LMSM. In direction of readout - First Exposure, Second Exposure, Third Exposure, Fourth Exposure. L is long exposure, M is middle exposure and S is short exposure

Node	Description
transitionGroups	List of seamless mode switch use cases that this sensor mode can support. Supported modes are NONE, BINCROP43, BINQCFA43, BINHDR43, BINCROP169, BINQCFA169, BINHDR169, SHDR1SHDR2, SHDR2SHDR3, SHDR1SHDR3, SHDRALL, TRANSITIONGROUPMAX Refer to imx686 sensor XML for details. `<transitionGroups>BINCROP43 BINCROP169 </transitionGroups>`
PDModeKey	This should be mapped to a valid PDAF Mode Key. Each PDAF Mode is given a user defined mode key to link a sensor mode to the correct PDAF mode. Driver: • Sensor driver: – ResolutionId: 0 – PDModeKey: 10 • PDAF driver: – ResolutionId: 1 – PDModeKey: 10 In this example, PDAF mode 1 corresponds to Sensor mode 0. The link between these modes is established using PDModeKey equal to 10. Max value of PDModeKey can be 99.

If INSENSORZOOM is set, then the resolution information should be the same. For example, assuming mode 0 resolutionInfo is 8000*6000, mode 9 is the corresponding mode with INSENSORZOOM, and then mode 9 resolutionInfo should be the same as mode 0 as there is no sensor reconfiguration during mode switch (meaning that sensor hardware PLL clock settings will remain the same in both modes).

frameDimension is based on the sensor full output resolution. For example, frameDim = (0,517,4208,2104), fullResolutionWidth =4208, and fullResolutionHeight =3120. Then 517 is invalid as 517*2+2104 is larger than the frame boundary 3120.

In most cases, share the resolution information below when facing issues:

```
<resolutionInfo>
   <resolutionData>
   <colorFilterArrangement>BAYER_RGGB</colorFilterArrangement>
   <vc range="[0,3]">0</vc>
   <dt>43</dt>
   <xStart>0</xStart>
   <yStart>0</yStart>
   <width>8000</width>
   <height>6000</height>
```

```
<bitWidth>10</bitWidth>
<type>IMAGE</type>
<lineLengthPixelClock>9440</lineLengthPixelClock>
<frameLengthLines>6074</frameLengthLines>
<minHorizontalBlanking>678</minHorizontalBlanking>
<minVerticalBlanking>69</minVerticalBlanking>
<outputPixelClock>1586910000</outputPixelClock>
<horizontalBinning>1</horizontalBinning>
<verticalBinning>1</verticalBinning>
<frameRate>30.00</frameRate>
<laneCount>3</laneCount>
<settleTimeNs>14</settleTimeNs>
<is3Phase>1</is3Phase>
```

exposureControlInfo

The exposure control information node contains exposure details such as maximum gain, maximum line count, and conversion formulas for gain manipulation.

Field	Description

exposureControlInfo	This node holds the information about the exposure details such as maximum gain, maximum line count, and conversion formulas for gain manipulation. Example:
	`<exposureControlInfo>`
	`<maxAnalogGain>8</maxAnalogGain>`
	`<maxDigitalGain>2</maxDigitalGain>`
	`<verticalOffset>20</verticalOffset>`
	`<maxLineCount>65515</maxLineCount>`
	`<realToRegGain>512-(512/realGain)</realToRegGain>`
	`<regToRealGain>512/(512-regGain)</regToRealGain>`
	`</exposureControlInfo>`

maxAnalogGain	Maximum analog gain supported by the sensor.
maxDigitalGain	Maximum digital gain supported by the sensor.
verticalOffset	Minimum offset to be maintained between the line count and frame length lines.
maxLineCount	Maximum line count supported by the sensor.
minLineCount	Minimum line count supported by the sensor.
middleMaxLineCount	Maximum line count supported by the sensor for middle exposure.
middleMinLineCount	Minimum line count supported by the sensor for middle exposure.
shortMaxLineCount	Maximum line count supported by the sensor for short exposure.
shortMinLineCount	Minimum line count supported by the sensor for short exposure.
realToRegGain	Real gain to register the gain equation. The equation must contain realGain in the equation.
regToRealGain	Register gain to real gain equation. The equation must contain regGain in the equation.

streamOnSettings

The `streamOnSettings` node contains the register settings to start streaming.

Field	Description

`streamOnSettings`	Sequence of register settings to configure the device to start streaming. Example:
	`<streamOnSettings>`
	`  <regSetting>`
	`  <registerAddr>0x0100</registerAddr>`
	`  <registerData>0x1</registerData>`
	`  <regAddrType range="[1,4]">2</regAddrType>`
	`  <regDataType range="[1,4]">1</regDataType>`
	`  <operation>WRITE</operation>`
	`  <delayUs>0</delayUs>`
	`  </regSetting>`
	`</streamOnSettings>`

`regSetting`	Register setting configuration. Contains: Register address, register data, register address type, register DT, operation, and delay in micro seconds

`streamOffSettings`	Sequence of register settings to configure the device to stop streaming.
	Configure the following settings to match this guideline and take the clock and data lanes to LP11. Example:
	`<streamOffSettings>`
	`  <regSetting>`
	`  <registerAddr>0x0100</registerAddr>`
	`  <registerData>0x0</registerData>`
	`  <regAddrType range="[1,4]">2</regAddrType>`
	`  <regDataType range="[1,4]">1</regDataType>`
	`  <operation>WRITE</operation>`
	`  <delayUs>0</delayUs>`
	`  </regSetting>`
	`</streamOffSettings>`

`regSetting`	Register setting configuration. Contains: Register address, register data, register address type, register DT, operation, and delay in micro seconds

groupHoldOnSettings

The `groupHoldOnSettings` node contains the register settings to configure the device in group hold settings.

<table>
<tr><th>Field</th><th colspan="2">Description</th></tr>
<tr><td><code>groupHoldOnSettings</code></td><td colspan="2">Sequence of register settings to configure the device in group hold settings. Example:
<code><groupHoldOnSettings>
  <regSetting>
    <registerAddr>0x0104</registerAddr>
    <registerData>0x1</registerData>
    <regAddrType range="[1,4]">2</regAddrType>
    <regDataType range="[1,4]">1</regDataType>
    <operation>WRITE</operation>
    <delayUs>0</delayUs>
  </regSetting>
</groupHoldOnSettings></code></td></tr>
<tr><td><code>regSetting</code></td><td colspan="2">Register setting configuration.
Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.</td></tr>
<tr><td><code>groupHoldOffSettings</code></td><td colspan="2">Sequence of register settings to configure the device to stop group hold settings. Example:
<code><groupHoldOffSettings>
  <regSetting>
    <registerAddr>0x0104</registerAddr>
    <registerData>0x0</registerData>
    <regAddrType range="[1,4]">2</regAddrType>
    <regDataType range="[1,4]">1</regDataType>
    <operation>WRITE</operation>
    <delayUs>0</delayUs>
  </regSetting>
</groupHoldOffSettings></code></td></tr>
<tr><td><code>regSetting</code></td><td>Register setting configuration.</td><td>Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.</td></tr>
</table>

initSettings

The `initSettings` node contains the sequence of register settings to initialize the sensor.

Field	Description

`initSettings`	Sequence of register settings to initialize the sensor.
	NOTE: Do not add stream on settings for the sensor in the initialization settings, as this enables the sensor streaming before real software stream on and may cause PHY to miss LP sequence.
	Example:
	`<initSettings>`
	`  <regSetting>`
	`  <registerAddr>0x136</registerAddr>`
	`  <registerData>0x18</registerData>`
	`  <regAddrType range="[1,4]">2</regAddrType>`
	`  <regDataType range="[1,4]">1</regDataType>`
	`  <operation>WRITE</operation>`
	`  <delayUs>0</delayUs>`
	`  </regSetting>`
	`</initSettings>`

`regSetting`	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

testPatternInfo

The `testPatternInfo` node contains test pattern generation register settings.

Field	Description
`testPatternInfo`	Contains the information about the test pattern generation register settings. Example:

```
<testPatternInfo>
  <testPatternData>
   <mode>OFF</mode>
   <settings>
     <regSetting>
       <registerAddr>0x136</
registerAddr>
       <registerData>0x18</
registerData>
       <regAddrType range="[1,4]
">2</regAddrType>
       <regDataType range="[1,4]
">1</regDataType>
       <operation>WRITE</
operation>
       <delayUs>0</delayUs>
     </regSetting>
   </settings>
  </testPatternData>
</testPatternInfo>
```

testPatternData

The `testPatternData` node contains register and mode settings for a particular test pattern.

Field	Description
testPatternData This node holds the register and mode settings of a particular test pattern.	
`Mode`	Supported modes are: • OFF • SOLID_COLOR • COLOR_BARS • COLOR_BARS_FADE_TO_GRAY • PN9 • CUSTOM1
`settings`	Sequence of register settings to configure the test pattern on the sensor.
`regSetting`	Register setting configuration. Contains: Register address, register data, register address type, register data type, operation, and delay in micro seconds.

colorLevelInfo

The color level information node contains details about the various channels in complete dark light.

Field	Description

`colorLevelInfo`	Color level information.
	The default currents in various channels in complete dark light. Example:
	`<colorLevelInfo>`
	`<whiteLevel>1023</whiteLevel>`
	`<rPedestal>64</rPedestal>`
	`<grPedestal>64</grPedestal>`
	`<bPedestal>64</bPedestal>`
	`<gbPedestal>64</gbPedestal>`
	`</colorLevelInfo>`

`whiteLevel`	White level value.
`rPedestal`	Pedestal value for the red channel.
`grPedestal`	Pedestal value for the green-red channel.
`bPedestal`	Pedestal value for the blue channel.
`gbPedestal`	Pedestal value for the green-blue channel.

opticalBlackRegionInfo

Field	Description

opticalBlackRegionInfo	Information about black regions. Multiple black regions are provided, if applicable. Example:
	`<opticalBlackRegionInfo>`
	`  <dimension>`
	`  <xStart>0</xStart>`
	`  <yStart>0</yStart>`
	`  <width>4608</width>`
	`  <height>2592</height>`
	`  </dimension>`
	`</opticalBlackRegionInfo>`

dimension	Frame dimension: Contains xStart, yStart, width, and height.
xStart	X start coordinate of the region.
yStart	Y start coordinate of the region.
width	Width of the region.
height	Height of the region.

pixelArrayInfo

Field	Description
pixelArrayInfo	Information about the pixel array. Active dimension and dummy pixels width are provided.

activeDimension	Width and height of the frame or subframe. Example:
	`<activeDimension>`
	`  <xStart>0</xStart>`
	`  <yStart>0</yStart>`
	`  <width>4608</width>`
	`  <height>2592</height>`
	`</activeDimension>`

xStart	Start of the X output in the frame.
yStart	Start of the Y output in the frame.
width	Total width of the frame in pixels.
height	Total height of the frame in pixels.

dummyInfo	Dummy pixels surrounding the active pixel array. Example:
	`<dummyInfo>`
	`  <left>0</left>`
	`  <right>0</right>`
	`  <top>0</top>`
	`  <bottom>0</bottom>`
	`</dummyInfo>`

left	Starting coordinate of the left dummy pixel.
right	Starting coordinate of the right dummy pixel.
top	Starting coordinate of the top dummy pixel.
bottom	Starting coordinate of the bottom dummy pixel.

delayInfo

Field	Description
`linecount`	Number of frames required to apply the line count.
`gain`	Number of frames required to apply the gain.
`maxPipeline`	Maximum pipeline delay in number of frames.
`frameSkip`	Number of initial bad frames to skip.
`frameLengthLines`	Number of frames required to apply frame length lines
`modeSwitch`	Number of frames required to apply mode switch settings
`virtualChannelSwitch`	Number of frames required to apply virtual channel settings

sensorProperty

Field	Description

`sensorProperty`	Sensor property information. Example:
	`<sensorProperty>`
	`<pixelSize>0</pixelSize>`
	`<cropFactor>0</cropFactor>`
	`<sensingMethod>ONE_CHIP_COLOR_AREA</sensingMethod>`
	`</sensorProperty>`

`pixelSize`	Pixel size in micro meters.
`cropFactor`	Crop factor.
`sensingMethod`	Sensing method of sensor. Supported sensing methods: • UNDEFINED • ONE_CHIP_COLOR_AREA TWO_CHIP_COLOR_AREA • THREE_CHIP_COLOR_AREA • COLOR_SEQUENCE_AREA • TRILINEAR • COLOR_SEQUENCE_LINEAR

frameSyncInfo

Field	Description

masterSettings	Sequence of register settings to configure the sensor to HW master mode. Valid only when HW sync is enabled. Example:
	`< masterSettings>`
	`<regSetting>`
	`<registerAddr>0x0350</registerAddr>`
	`<registerData>0x0</registerData>`
	`<regAddrType range="[1,4]">2</regAddrType>`
	`<regDataType range="[1,4]">1</regDataType>`
	`<operation>WRITE</operation>`
	`<delayUs>0</delayUs>`
	`</regSetting>`
	`</ masterSettings>`

slaveSettings	Sequence of register settings to configure the sensor to HW slave mode. It is similar to masterSettings.

softwareSyncConfiguration	Flag to indicate if we need to compensate blanking for current sensor. It is only valid when SW frame sync is enabled and it is only valid on the SM8350 platform.
	Refer to KBA-200927214121 for more details. Example:
	`< softwareSyncConfiguration>`
	`<blankingCompensation>true</blankingCompensation>`
	`</softwareSyncConfiguration>`

Mode switch register information

Field	Description

modeSwitchRegInfo	Register settings and virtual channel register addresses to be programmed during seamless mode switch for the following data types:
	• PDAFVCAddress
	• ImageLongVCAddress
	• ImageShortVCAddress
	• ImageMiddleVCAddress
	• BlobVCAddress
	• HDRVCAddress
	• MetaVCAddress
	Example:
	`<modeSwitchRegInfo>`
	`<regAddrType>2</regAddrType>`
	`<regDataType>1</regDataType>`
	`<ImageLongVCAddress>0x0110</ImageLongVCAddress>`
	`<PDAFVCAddress>0x3076</PDAFVCAddress>`
	`</modeSwitchRegInfo>.`

Register setting group information

Field	Description
registerSettings	Register settings that need to be programmed as a part of this group.
registerSettingsType	Specifies the register setting type. Supported values are: • StreamOn - Indicates streamon settings for special use cases • StreamOff - Indicates streamoff settings for special use cases.
registerSettingsFlag	Indicates any settings specially applied for certain use cases. Supported values are: • PN9Test - This indicates settings to be used during PN9 Pattern testing case.

Module configuration XML

The module configuration XML stores camera module-specific information, such as the lens information, mount angles, actuator, OIS, and flash-related information.

CameraPosition

Field	Description
`CameraPosition`	Position of the sensor module. Supported values are: • REAR • FRONT • REAR_AUX • FRONT_AUX • EXTERNAL

LensInformation

Field	Description
`LensInformation`	This node holds sensor lens-specific information.
`focalLength`	Focal length of the lens in millimeters.
`fNumber`	F-Number of the optical system.
`minFocusDistance`	Minimum focus distance in meters.
`maxFocusDistance`	Total focus distance in meters.
`horizontalViewAngle`	Horizontal view angle in degrees.
`verticalViewAngle`	Vertical view angle in degrees.
`maxRollDegree`	Maximum roll degree.
`maxPitchDegree`	Maximum pitch degree.
`maxYawDegree`	Maximum yaw degree.

LensInformationList

Field	Description
`LensInformationList`	This node holds sensor list of lens-specific information based on version number.
`version`	This corresponds to module version defined in EEPROM.
`chromatixName`	Chromatix name corresponding to this version.
`lensInfo`	Lens specific information

ModuleConfiguration

Field	Description
ModuleConfiguration	Module configuration.
cameraId	cameraId is the ID to which DTSI node is mapped. Typically, cameraId is the slot ID for Non-Combo mode.
moduleName	Name of the module integrator.
sensorName	Name of the sensor in the image sensor module.
sensorSlaveAddress	Sensor slave address to override the value present in the binary. Optional element. Do not use this entry if override is not required.
sensorI2CFrequencyMode	Sensor operation frequency mode. Supported types are: • STANDARD • FAST • FAST_PLUS • CUSTOM
actuatorName	Name of the actuator in the image sensor module. Optional element. Skip this element if an actuator is not present.
oisName	Name of the OIS in the image sensor module. Optional element. Skip this element if OIS is not present.
eepromName	Name of the EEPROM in the image sensor module. Optional element. Skip this element if EEPROM is not present.
flashName	Flash name to open binary. Binary name is of form flashName_flash.bin Ex: pmic_flash.bin.
chromatixName	Open binary. Binary name is of the form sensor_model_chromatix.bin.
position	Camera position. Supported types are: • REAR • FRONT • REAR_AUX • FRONT_AUX • EXTERNAL
CSIInfo	This holds CSI information such as lane assign, combo mode, and cphy-dphy combo mode flags
laneAssign	Indicates the value used to determine the CPHY and DPHY lanes that should be assigned to this sensor. Example: 0x2310
isComboMode	Flag to enable Combo mode. This flag is enabled if multiple sensors are using the same CSI-PHY receiver: • First camera – Connected to PHY lanes 2:0. Clock lane is connected to PHY lane

Field	Description
`ModuleGroup`	Module group can contain either one module or two modules. Dual camera, stereo camera use cases contain two modules in the group.
`moduleConfiguration`	Module configuration information. See ModuleConfiguration.

Field	Description
`CameraModuleData`	Camera module data.
`module_version`	Camera module version.
`major_revision`	Specifies the driver revision.
`minor_revision`	Specifies the minor revision number.
`incr_revision`	Specifies incremental revision number.
`moduleGroup`	Specifies the module configuration data (see ModuleGroup) count and ID.

Field	Description
`remoteLaneAssign`	Contains value of remote lane assigned
`remoteIsComboMode`	Flag to enable combo mode. This flag is enabled if multiple sensors are using same CSI-PHY receiver.
`remoteCSIPHYID`	Contains remote CSIPHY ID

For example:

```
<cameraId>4</cameraId>
<!--Name of the module integrator -->
<moduleName>semco</moduleName>
<!--Name of the sensor in the image sensor module -->
<sensorName>imx586</sensorName>
<!--Actuator name in the image sensor module
This is an optional element. Skip this element if actuator is not
present -->
<actuatorName>lc898217xc</actuatorName>
<eepromName>cat24c64_imx586</eepromName>
<flashName>pmic</flashName>
<!--Chromatix name is used to used to open binary.
Binary name is of the form sensor_model_chromatix.bin -->
<chromatixName>semco_imx586</chromatixName>
```

2.2 Sensor hardware configuration

The hardware configuration of the camera sensor module is saved in the kernel DTSI files.

Refer to `kernel/msm-4.19/Documentation/devicetree/bindings/media/camera/`

for more information about how to customize camera DTSI.

Sensor kernel nodes

Descriptions of the various elements of the Camera_LDO table, CCI-based sensor driver table, OIS node, and EEPROM node.

Camera_LDO

Field	Description
compatible	Data type to which this node corresponds.
reg	ID of the node.
regulator-name	LDO regulator name.
regulator-min-microvolt	Minimum voltage for this LDO.
regulator-max-microvolt	Maximum voltage for this LDO.
regulator-enable-ramp-delay	The time taken, in microseconds, for the supply rail to reach the target voltage, ± whatever tolerance the board design requires. This property describes the total system ramp time required due to the combination of internal ramping of the regulator itself, and board design issues such as trace capacitance and load on the supply.
enable-active-high	Enable with active high.
gpio	GPIO to be used with this node.
pinctrl-names	Pinctrl name.
pinctrl-0	Pinctrl handle for this GPIO.
vin-supply	Input voltage supply node name.

For example:

```
camera_ldo: gpio-regulator@2 { compatible = "regulator-fixed"; reg =
<0x02 0x00>;
   regulator-name = "camera_ldo"; regulator-min-microvolt = <1050000>
;
   regulator-max-microvolt = <1050000>;
   regulator-enable-ramp-delay = <233>; enable-active-high;
   gpio = <&pm8998_gpios 9 0>; pinctrl-names = "default";
   pinctrl-0 = <&camera_dvdd_en_default>; vin-supply = <&pm8998_s3>;
```

```
    };
```

CCI-based sensor driver

Field	Description
cell-index	Points to the ID of the sensor node.
reg	Corresponds to the cell-index of the node.
compatible	Data type to which this node corresponds to.
qcom,csiphy-sd-index	Paired CSIPHY node for this DT.
qcom,sensor-position-roll	Sensor position roll angle.
qcom,sensor-position-pitch	Sensor position pitch angle.
qcom,sensor-position-yaw	Sensor position yaw angle.
qcom,led-flash-src	LED flash source node name.
qcom,actuator-src	Actuator source node name.
qcom,ois-src	OIS source node name.
qcom,eeprom-src	EEPROM source node name.
cam_vio-supply	I/O voltage supply source.
cam_vana-supply	Analog voltage source.
cam_vdig-supply	Digital voltage source.
cam_clk-supply	Camera clock source.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
regulator-names	Should contain names of all regulators needed.
rgltr-cntrl-support	This property is required if the SW control regulator parameters (e.g., rgltr-min-voltage).
pwm-switch	This property is required for regulator to switch into PWM mode.
rgltr-min-voltage	Should contain minimum voltage level for regulators mentioned in regulator-names property (in the same order).
rgltr-max-voltage	Should contain maximum voltage level for regulators mentioned in regulator-names property (in the same order).
rgltr-load-current	Should contain optimum voltage level for regulators mentioned in regulator-names property (in the same order).
qcom,gpio-no-mux	Whether GPIO muxing is enabled.
pinctrl-names	Pinctrl handle names for this GPIO if target uses pinctrl.

Field	Description
pinctrl-0	Binding to GPIO pin and function node.
pinctrl-1	Binding to GPIO pin and function node.
gpios	List of GPIO pins used.
qcom,gpio-reset	Should contain index to GPIO used by sensors reset_n.
qcom,gpio-vana	Should contain index to GPIO used by sensors analog vreg enable.
qcom,gpio-vaf	Should contain index to GPIO used by sensors of vreg enable.
qcom,gpio-req-tbl-num	Should contain index to GPIO specific to this sensor.
qcom,gpio-req-tbl-flags	Should contain direction of GPIO present in qcom, gpio- req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Should contain name of GPIO present in qcom, gpio-req- tbl-num property (in the same order).
qcom,sensor-position	Mount angle of the sensor • 0 – Back Camera • 1 – Front Camera
qcom,sensor-mode	Supported sensor mode • 0 – back camera 2D • 1 – front camera 2D • 2 – back camera 3D • 3 – back camera int 3D
qcom,cci-master	I2C master used for this sensor • 0 – MASTER 0 • 1 – MASTER 1
status	Whether this node is enabled or disabled.
clocks	Clock node names.
clock-names	Name of the clocks required for the device.
qcom,clock-rates	Clock rate in Hz.

For example:

```
qcom,cam-sensor@0 { cell-index = <0>;
   compatible = "qcom,cam-sensor"; reg = <0x0>;
   qcom,csiphy-sd-index = <0>;
   qcom,sensor-position-roll = <90>;
   qcom,sensor-position-pitch = <0>;
```

```
   qcom,sensor-position-yaw = <180>; qcom,led-flash-src = <&led_
flash_rear>; qcom,actuator-src = <&actuator_rear>; qcom,ois-src = <&
ois_rear>;
   qcom,eeprom-src = <&eeprom_rear>; cam_vio-supply = <&pm8998_lvs1>;
   cam_vana-supply = <&pmi8998_bob>; cam_vdig-supply = <&camera_rear_
ldo>; cam_clk-supply = <&titan_top_gdsc>;
   qcom,cam-vreg-name = "cam_vio", "cam_vana", "cam_vdig", "cam_clk";
   qcom,cam-vreg-min-voltage = <0 3312000 1050000 0>;
   qcom,cam-vreg-max-voltage = <0 3600000 1050000 0>;
   qcom,cam-vreg-op-mode = <0 80000 105000 0>;
   qcom,gpio-no-mux = <0>;
   pinctrl-names = "cam_default", "cam_suspend";
   pinctrl-0 = <&cam_sensor_mclk0_active
      &cam_sensor_rear_active>;
   pinctrl-1 = <&cam_sensor_mclk0_suspend
      &cam_sensor_rear_suspend>; gpios = <&tlmm 13 0>,
      <&tlmm 80 0>,
      <&tlmm 79 0>;
   qcom,gpio-reset = <1>;
   qcom,gpio-vana = <2>;
   qcom,gpio-req-tbl-num = <0 1 2>;
   qcom,gpio-req-tbl-flags = <1 0 0>;
   qcom,gpio-req-tbl-label = "CAMIF_MCLK0",
      "CAM_RESET0",
      "CAM_VANA";
   qcom,sensor-mode = <0>;
   qcom,cci-master = <0>;
   status = "ok";
   clocks = <&clock_camcc CAM_CC_MCLK0_CLK>;
   clock-names = "cam_clk";
   qcom,clock-rates = <24000000>;
};
```

Field	Description
cell-index	Points to the ID of this node.
reg	Corresponds to the cell-index of the node.
compatible	Data type to which this node corresponds to.
qcom,cci-master	CCI node that drives this node.
cam_vaf-supply	Should contain regulator from which AF voltage is supplied.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
status	Whether this node is enabled or disabled.

For example:

```
ois_rear: qcom,ois@0 { cell-index = <0>; reg = <0x0>;
   compatible = "qcom,ois"; qcom,cci-master = <0>;
   cam_vaf-supply = <&actuator_regulator>; qcom,cam-vreg-name =
   "cam_vaf"; qcom,cam-vreg-min-voltage = <2800000>;
   qcom,cam-vreg-max-voltage = <2800000>;
   qcom,cam-vreg-op-mode = <0>; status = "disabled";
};
```

EEPROM node

Field	Description
cell-index	Points to the node ID.
reg	Corresponds to the cell-index of the node.
compatible	Data type that this node corresponds to.
qcom,cci-master	CCI node that drives this node.
cam_vio-supply	Input/output voltage supply source.
cam_vana-supply	Analog voltage source.
cam_vdig-supply	Digital voltage source.
cam_clk-supply	Camera clock source.
qcom,cam-vreg-name	Voltage regulator node names.
qcom,cam-vreg-min-voltage	Minimum voltage, in the above order.
qcom,cam-vreg-max-voltage	Maximum voltage, in the above order.
qcom,cam-vreg-op-mode	Operation mode of the voltage regulator, in above order.
qcom,gpio-no-mux	Whether GPIO muxing is enabled.
pinctrl-names	Pinctrl handle names.

Field	Description
pinctrl-0	Binding to GPIO pin and function node.
pinctrl-1	Binding to GPIO pin and function node.
gpios	List of GPIO pins used.
qcom,gpio-reset	Should contain index to GPIO used by sensors reset_n.
qcom,gpio-vana	Should contain index to GPIO used by sensors analog vreg enable.
qcom,gpio-vaf	Should contain index to GPIO used by sensors i2af vreg enable.
qcom,gpio-req-tbl-num	Should contain index to GPIO specific to this sensor.
qcom,gpio-req-tbl-flags	Should contain direction of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Should contain name of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,sensor-position	Mount angle of the sensor • 0 - Back camera • 1 - Front camera
qcom,sensor-mode	Supported sensor mode • 0 - Back camera 2D • 1 - Front camera 2D • 2 - Back camera 3D • 3 - Back camera int 3D
qcom,cci-master	I2C master used for this sensor • 0 - MASTER 0 • 1 - MASTER 1
status	Whether this node is enabled or disabled.
clocks	Clock node names.
clock-names	Name of the clocks required for the device.
qcom,clock-rates	Clock rate in Hz.

For example:

```
eeprom_rear: qcom,eeprom@0 { cell-index = <0>;
   reg = <0>;
   compatible = "qcom,eeprom"; cam_vio-supply = <&pm8998_lvs1>; cam_
vana-supply = <&pmi8998_bob>;
   cam_vdig-supply = <&camera_rear_ldo>; cam_clk-supply = <&titan_
```

```
top_gdsc>;
   qcom,cam-vreg-name = "cam_vio", "cam_vana", "cam_vdig", "cam_clk";
   qcom,cam-vreg-min-voltage = <0 3312000 1050000 0>;
   qcom,cam-vreg-max-voltage = <0 3600000 1050000 0>;
   qcom,cam-vreg-op-mode = <0 80000 105000 0>;
   qcom,gpio-no-mux = <0>;
   pinctrl-names = "cam_default", "cam_suspend";
   pinctrl-0 = <&cam_sensor_mclk0_active &cam_sensor_rear_active>;
   pinctrl-1 = <&cam_sensor_mclk0_suspend &cam_sensor_rear_suspend>;
   gpios = <&tlmm 13 0>,
      <&tlmm 80 0>,
      <&tlmm 79 0>,
      <&tlmm 27 0>;
   qcom,gpio-reset = <1>;
   qcom,gpio-vana = <2>;
   qcom,gpio-vaf = <3>;
   qcom,gpio-req-tbl-num = <0 1 2 3>;
   qcom,gpio-req-tbl-flags = <1 0 0 0>;
   qcom,gpio-req-tbl-label = "CAMIF_MCLK0", "CAM_RESET0", "CAM_VANA0
", "CAM_VAF";
   qcom,sensor-position = <0>;
   qcom,sensor-mode = <0>;
   qcom,cci-master = <0>; status = "ok";
   clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
   qcom,clock-rates = <24000000>;
};
```

CCI timing and debug

The I2C timing may vary depends on how many slave devices on the I2C bus, ensure the characteristics of the SDA and SCL I/O stages (High to Low, Low to High, etc.) are within the specified range described in https://www.nxp.com/docs/en/user-guide/UM10204.pdf

Review the table below for the most common CCI errors.

IRQ status	Error reason	What to check
NACK_ERR	I2C command of I2C Master NACKed due to no ACK response from slave.	Check if any Slave devices are holding the bus due to improper power on/down, or board level noise.
CMD_ERR	An unsupported command word is programmed into HW.	Check if any Slave devices are holding the bus due to improper power on/down, or board level noise.
OVERFLOW	I2C command FIFO or I2C RD FIFO has overflowed.	Ensure we do not load command words more than the available space in I2C Master command FIFO.
UNDERFLOW	I2C RD FIFO has underflowed.	Ensure we do not read from I2C master RD FIFO when it is empty.

Follow the adb instructions below to enable CCI register dump CCI_REGISTERS

```
adb shell "echo 0xF > /sys/kernel/debug/cam_cci/en_dump_cci0"
adb shell "echo 0xF > /sys/kernel/debug/cam_cci/en_dump_cci1"
```

For CCI register interpretation, please review 80-PL546-2X.

Common CCI error log:

```
if (irq_status0 & CCI_IRQ_STATUS_0_I2C_M0_ERROR_BMSK) { cci_dev->cci_
master_info[MASTER_0].status = -EINVAL;
if (irq_status0 & CCI_IRQ_STATUS_0_I2C_M0_Q0_NACK_ERROR_BMSK) {CAM_
ERR(CAM_CCI, "Base:%pK, M0_Q0 NACK ERROR: 0x%x",
base, irq_status0);
complete_all(&cci_dev->cci_master_info[MASTER_0]
.report_q[QUEUE_0]);
}
if (irq_status0 & CCI_IRQ_STATUS_0_I2C_M0_Q1_NACK_ERROR_BMSK) {CAM_
ERR(CAM_CCI, "Base:%pK, M0_Q1 NACK ERROR: 0x%x",
base, irq_status0);
complete_all(&cci_dev->cci_master_info[MASTER_0]
.report_q[QUEUE_1]);
}
if (irq_status0 & CCI_IRQ_STATUS_0_I2C_M0_Q0Q1_ERROR_BMSK) CAM_
```

```
ERR(CAM_CCI, "Base:%pK, M0 QUEUE_OVER/UNDER_FLOW OR CMD ERR: 0x%x",
base, irq_status0);
if (irq_status0 & CCI_IRQ_STATUS_0_I2C_M0_RD_ERROR_BMSK) CAM_ERR(CAM_
CCI, "Base: %pK, M0 RD_OVER/UNDER_FLOW ERROR: 0x%x",
```

Configure CCI operation speed

How to set the I2C Frequency mode, which controls the operation speed.

Following the I2C specification, CCI (for chipsets where it is present) can operate on the following frequencies:

- 100 kHz (STANDARD)
- 400 kHz (FAST)
- 1 MHz (FAST PLUS)

I2C Frequency mode controls the operation speed. For Custom mode, the CCI tuning parameters setting information is in `arch/arm64/boot/dts/vendor/qcom/camera/XXX-camera.dtsi`.

For SM8450 and onwards refer to `vendor\qcom\proprietary\camera-devicetree\XXX-camera.dtsi`.

Example:

```
i2c_freq_custom: qcom,i2c_custom_mode { qcom,hw-thigh = <38>;
   qcom,hw-tlow = <56>;
   qcom,hw-tsu-sto = <40>;
   qcom,hw-tsu-sta = <40>;
   qcom,hw-thd-dat = <22>;
   qcom,hw-thd-sta = <35>;
   qcom,hw-tbuf = <62>;
   qcom,hw-scl-stretch-en = <1>;
   qcom,hw-trdhld = <6>;
   qcom,hw-tsp = <3>;
   qcom,cci-clk-src = <37500000>; status = "ok";
};
```

If a custom CCI configuration is used, then the speed of I2C frequency information is calculated by the formula `CCI clock = (src clock)/(hw_thigh+ hw_tlow)`. Because the CCI clock frequency is typically 19.2 MHz, the standard CCI frequency case is:

`CCI clock = 19.2 MHz/(78 + 114) = 100 kHz`

If there is no CCI hardware, or when QUP is used for I2C, QUP speed is set to either 100 kHz or 400 kHz from the DTSI file. The recommendation is to use one of the supported operation

frequencies instead of custom mode as they have been fully verified.

Using interfaces other than I2C

The serial peripheral interface (SPI) is not supported for sensor configuration. It only supports the EEPROM driver.

For the BLSP configuration, see *BAM Low-Speed Peripherals for Linux Kernel Configuration and Debugging Guide* (80-NE436-1). For chipsets that do not have CCI hardware, QUP is used to perform I2C transactions with camera.

NOTE Customers who need to configure QUP/SPI on chipsets with CCI hardware must contact QTI.

IFE blanking requirements

The following are the IFE blanking requirements for the ISP.

For SM8150:

- Minimum horizontal blanking (pixels) – 64
- Minimum vertical blanking (lines) – 32

For SM8250 and SM8350:

- Minimum horizontal blanking (pixels) - 128
- Minimum vertical blanking (lines) - 36 for 720p input or higher

For SM8450, SM8550 and SM8650:

- Minimum horizontal blanking (pixels)
  - 128 for non HVX
  - 160 for HVX use cases
- Minimum vertical blanking (lines)
  - 90 if SFE is enabled
  - Max (MinVBI_default, MinVBI_stats) if SFE is not enabled.

    Where:
  - MinVBI _stats
    - MinVBI _stats= StatsFlushCycles/(LineTime*IFE_Freq_min) + SensLinesFlush4Stats
    - StatsFlushCycles= 8448
    - SensLinesFlush4Stats= 31 for nonHvx, 41 if HVX is enabled

- MINVBI_default
  - MINVBI_default= 55+7us/LineTime
  - LineTime = 1/FPS/(sensorHeight + sensorVBI)

For SM8550, the following logs could provide the minimum horizonal and vertical blanking information:

```
02-12 02:54:45.582 7086 7108 I CamX : [ INFO][POWER ] camxifeutils.
cpp:1117 CalculatePixelClockRate() inputWidth=2048 SensorMinHBI=1534
SensorMinVBI=11722 IFEminHBI=64 IFEminVBI=80 SensorOpCLK 1783950000
SensorResolution=2048X1536 SFEFactor=1.000000
```

For SM8750 and SM8550:

- Minimum horizontal blanking(pixels) - 96
- Minimum vertical blanking (lines) - 64 for all usecases where input frame resolution >= 1 MP

Camera IFE clock configuration

This section discusses how to optimally configure the IFE clock for SM8150 and SM8250.

Factors affecting IFE clock

The IFE clock rate is determined by the following sensor-based factors:

- Input frame dimensions for the sensor mode in use. Input dimensions are specified in terms of the pixel width and height of the frame being fed into the IFE from the sensor.
- The horizontal and vertical blanking periods for the sensor mode in use.
- The sensor output clock rate for the sensor mode in use.

The input dimensions and the blanking periods are specified in the corresponding sensor's XML configuration file. For example, a typical sensor may have a configuration for a particular mode that may look like this:

```
<resolutionData>
<streamInfo>
<streamConfiguration>
<vc range="[0,3]">0</vc>
<dt>43</dt>
<frameDimension>
<xStart>0</xStart>
<yStart>0</yStart>
<width>5488</width>
<height>4112</height>
```

```
</frameDimension>
<!--Minimum horizontal blanking interval in terms of the sensor's
output pixel clock rate -->
<minHorizontalBlanking>467</minHorizontalBlanking>
<!--Minimum vertical blanking interval in terms lines -->
<minVerticalBlanking>278</minVerticalBlanking>
<outputPixelClock>784800000</outputPixelClock>
...
```

The frame width, height, blanking information, and sensor output pixel clock rate are provided by the sensor vendor for each sensor mode. The blanking information provided by sensor vendors may not be the minimum blanking period, but the average blanking period. To calculate the IFE clock rate optimally and correctly, we cannot use the average blanking period as this can result in CAMIF overflows. Instead, the smallest horizontal and vertical blanking periods for a given frame must be specified.

Configuring the correct blanking values

Most of the time, the minHorizontalBlanking and minVerticalBlanking values will be provided by the sensor vendor. These values are used internally for calculating the correct IFE clock frequency and bandwidth clock frequency to process the sensor frame data in the IFE/TFE.

Sometimes, the sensor vendor does not provide the smallest blanking period in their data sheets. In such cases, we can use the Spectra HW's ability to measure the smallest horizontal blanking interval (HBI) and the smallest vertical blanking interval (VBI) in the CSID block to get the correct values. Note that the CSID measures the HBI and VBI values during sensor data streaming in the CSID. Using the measured values, minHorizontalBlanking and minVerticalBlanking could be calculated and are used for sensor xml configuration.

To measure HBI/VBI values in the CSID:

1. Set the CSID clock to max clock (480 MHz):

```
adb shell "echo csidClockFrequencyMHz=0xFFFFFFFF >> /vendor/etc/
camera/camxoverridesettings.txt"
```

> **Note:** From SM8450 onwards, the CSID clock will be set either 400M or 480M. In SM8750, the CSID clock will be set to either 266M, 400M, or 480M depending on how many clients are running and the calculated CSID clock setting in UMD. Since the CSID clock cannot be read while the sensor is streaming, customers need to set the CSID clock to max to print CSID HBI/VBI cycles.

2. Get the KMD log:

```
adb shell "echo 0x80 > /sys/kernel/debug/camera_ife/ife_csid_
debug"

adb shell "echo 0x1 > /sys/kernel/debug/camera_ife/ife_camif_
debug"

adb logcat -b kernel > kmd.log
```

Note: csid_debug and camif debug masks need to be enabled to print blanking cycles.

---

3. Search for keyword.

   a. Before SM8450, the keyword is `cam_ife_csid_get_hbi_vbi`. Read the printed values (hex) for HBI and VBI in the log with the keyword.

      - For HBI, get only 12-bit LSB values** (for example, 0x16d in HBI: 0x665016d)
      - For VBI, get 32-bit LSB values (for example, 0x32e98 in VBI: 0x32e98)

   The following is an example on the SM8350 chipset:

```
07-25 03:52:51.427 0 0 W **cam_ife_csid_get_hbi_vbi**: 292
callbacks suppressed

07-25 03:52:51.427 0 0 I CAM_INFO: CAM-ISP: cam_ife_csid_get_
hbi_vbi: 3090 Device csid4 index 0 Resource 4 HBI: 0x665016d
VBI: 0x32e98 07-25 03:52:51.427 0 0 I CAM_INFO: CAM-ISP: cam_
ife_csid_get_hbi_vbi: 3090 Device

csid4 index 1 Resource 4 HBI: 0x665016d VBI: 0x32e98 07-25
03:52:51.461 0 0 I CAM_INFO: CAM-ISP: cam_ife_csid_get_hbi_
vbi: 3090 Device csid4 index 0

Resource 4 HBI: 0x665016d VBI: 0x32e99 07-25 03:52:51.461 0 0
I CAM_INFO: CAM-

ISP: cam_ife_csid_get_hbi_vbi: 3090 Device csid4 index 1
Resource 4 HBI: 0x665016d VBI: 0x32e99 07-25 03:52:51.494 0 0
I CAM_INFO: CAM-ISP:

**cam_ife_csid_get_hbi_vbi**: 3090 Device csid4 index 0
Resource 4 **HBI: 0x665016d VBI: 0x32e98** 07-25 03:52:51.494
0 0 I CAM_INFO: CAM-ISP:
```

```
cam_ife_csid_get_hbi_vbi: 3090 Device csid4 index 1 Resource
4 HBI: 0x665016d VBI: 0x32e98 07-25 03:52:51.527 0 0 I CAM_
INFO: CAM-ISP:

cam_ife_csid_get_hbi_vbi: 3090 Device csid4 index 0 Resource
4 HBI: 0x665016d VBI: 0x32e98 07-25 03:52:51.527 0 0 I CAM_
INFO: CAM-ISP:

cam_ife_csid_get_hbi_vbi: 3090 Device csid4 index 1 Resource
4 HBI: 0x665016d VBI: 0x32e98
```

b. From SM8450 onward, the keyword is `cam_ife_csid_ver2_print_hbi_vbi`. Read the printed values (decimal or hex) for HBI and VBI in the log with the keyword.

- For HBI, min and max are printed in square brackets. You need to pick up the minimum value in the square bracket.
- For VBI, the 32 bits represented in hex needs to be converted to decimal. Decimal value needs to be used in calculations.

The following is an example on the SM8750 chipset:

```
12-18 11:47:52.795 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_
csid_ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:5, name:
IPP_0, hbi: 85525780[**1305, 1300**] cycles, **vbi: 670658**
cycles]

12-18 11:47:52.795 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_
csid_ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:6, name:
PPP, hbi: 268374015[4095, 4095] cycles, vbi: 670658 cycles]

12-18 11:47:52.802 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_
csid_ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:0, name:
RDI_0, hbi: 85525780[1305, 1300] cycles, vbi: 670658 cycles]

12-18 11:47:52.802 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_
csid_ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:1, name:
RDI_1, hbi: 4095[0, 4095] cycles, vbi: 0 cycles]

12-18 11:47:52.802 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_
csid_ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:2, name:
RDI_2, hbi: 268374015[4095, 4095] cycles, vbi: 847037 cycles]
```

As seen in the above example, there may be some small variation in the measured HBI and VBI values in the CSID. It is very important to always use the smallest values observed over many seconds. Also note that this measurement must be done with the

highest FPS supported by each sensor's mode. To ensure the highest FPS, ensure the device is being tested in a well-lit bright light environment and verify that the highest FPS is being achieved. The Spectra HW reports the measured HBI and VBI intervals in terms of CSID clock rate.

From SM8150 onwards for the HBI, only bits 0 to 11 should be used to extract the minimum HBI. So from the log, the minimum HBI is 0x16d. Convert this to the sensor's output pixel clock domain using the following formula:

$$minHorizontalBlanking = RoundUp(OutputPixClkRate * csidHBICyles/CSIDclockRate)$$

- OutputPixClkRate is the sensor's output pixel clock rate in Hz.
- csidHBIcycles is the HBI value as reported by the Spectra HW. In the above logs, this value is 0x16d.
- CSIDclockRate is the clock rate of the CSID block being used for this use case. Assume that it is always max clock (480M) when HBI and VBI cycles are measured.

Note: From SM8650 onward, customer could not get a running CSID clk because the camera clk source was moved to the cesta driver. Prior to SM8650, CSIDclockRate can be read using the following commands while the use case is active:

```
adb shell cat /d/clk/cam_cc_ife_0_csid_clk/clk_measure adb shell
cat /d/clk/cam_cc_ife_1_csid_clk/clk_measure
```

To calculate the minimum VBI, use the following formula:

$$minVerticalBlanking = ((csidVBICycles/CSIDclockRate) * height)/((1/fps) - (csidVBICycles/CSID$$

Note: The csidHBICyles and csidVBICycles values need to be picked up on the image path (IPP or RDI0) because the OutputPixClkRate and height values are matched to the image frame.

Refer to the following example from SM8750:

```
12-18 11:47:52.795 0 0 I [ C0] CAM_ERR: CAM-ISP: cam_ife_csid_
ver2_print_hbi_vbi: 7750: CSID[2] Resource[id:5, name:IPP_0,
hbi: 85525780[1305, 1300] cycles, vbi: 670658 cycles]
```

- CSIDclockRate = 480000000 (480 Mhz)
- Sensor Op = 1889664000 = 189M
- csid_HBI_cycles = 1300

- csid_VBI_cycles = 670658
- sensorOutputWidth = 4080
- sensorOutputHeight = 3060
- minHorizontalBlanking=5118
- minVerticalBlanking=280

Note: The measured minHorizontalBlanking and minVerticalBlanking for sensor XML configuration should meet the IFE blanking requirements provided in IFE blanking requirements.

—

Frequently asked questions

After setting the `minHorizontalBlanking` and `minVerticalBlanking` based on instruction in preceeding sections of this document, the IFE clock rate seems to be too high. Can the IFE clock rate be optimized?

The output pixel clock rate utilized by the vendor may be higher than the data rate needed to transfer pixel data alone. The reasons for this is only known to the sensor vendor. A somewhat crude and risky way to check if the IFE can operate at a lower rate is by checking if it still works by forcing a lower clock rate than the value calculated based on `minHorizontalBlanking` and `minVerticalBlanking`. This can be done by using the setting `ifeClockFrequencyMHz`:

```
# First determine available rates (shown in Hz)
adb root && adb wait-for-device remount && adb wait-for-device adb
shell cat /d/clk/cam_cc_ife_0_clk_src/clk_list_rates
# Sample output on SM8150 when the above command is run: 400000000
558000000
637000000
847000000
950000000
# To force the clock to 558MHz, do this:
adb shell "echo ifeClockFrequencyMHz=558 >> /vendor/etc/camera/
camxoverridesettings.txt"
adb reboot
```

Now try to run the camera with the sensor mode required. If CAMIF violation is reported in the KMD log (sample log below), then the clock rate is too low and a higher clock rate should be used.

```
12-14 03:14:01.854   0  0  E CAM_ERR : CAM-ISP: cam_ife_csid_irq:
3188 CSID:1
```

```
IPP fifo over flow
12-14 03:14:01.887   0   0   E CAM_ERR : CAM-ISP: cam_vfe_irq_err_top_
half: 176
Encountered Error:  vfe:0: Irq_status0=0x0 Status1=0x80
12-14 03:14:01.898   0   0   E CAM_ERR : CAM-ISP: cam_vfe_irq_err_top_
half: 179
Stopping further IRQ processing from this HW index=0
12-14 03:14:01.909   0   0 I CAM_INFO: CAM-ISP: cam_vfe_irq_err_top_
half: 213 Violation status = 1
12-14 03:14:01.916   0   0 E CAM_ERR : CAM-ISP:
cam_vfe_bus_error_irq_top_half: 2495 Bus Err IRQ
```

If for example, 558MHz works and the calculated clock rate was 600MHz, we can try lowering the `minHorizontalBlanking` and `minVerticalBlanking` so that calculated rate matches 558MHz using the formulas above.

Configure power regulators

The type of voltage to be applied is specified in driver XML for a given sensor. The supported values are:

- MCLK – mclock
- VANA – Analog supply voltage
- VDIG – Digital supply voltage
- VIO – I/O supply voltage
- VAF – Actuator supply voltage
- RESET – Reset GPIO
- STANDBY – Standby GPIO

The values maps to specific voltage regulators using the DTSI sensor node.

PMIC-based sensor nodes

For example, VIO maps to cam_vio-supply, VANA maps to cam_vana-supply, and MCLK maps to cam_clk-supply.

```
cam_vio-supply = <&pm8998_lvs1>; cam_vana-supply = <&pmi8998_bob>;
cam_vdig-supply = <&camera_rear_ldo>; cam_clk-supply =
<&titan_top_gdsc>;
```

GPIO-based sensor nodes

```
pinctrl-names = "cam_default", "cam_suspend";
pinctrl-0 = <&cam_sensor_mclk0_active &cam_sensor_rear_active>;
pinctrl-1 = <&cam_sensor_mclk0_suspend &cam_sensor_rear_suspend>;
gpios = <&tlmm 13 0>,
     <&tlmm 80 0>,
     <&tlmm 79 0>;
gpio-reset = <1>;
gpio-vana = <2>;
gpio-req-tbl-num = <0 1 2>;
gpio-req-tbl-flags = <1 0 0>;
gpio-req-tbl-label = "CAMIF_MCLK0","CAM_RESET0","CAM_VANA";
```

Regulator-based sensor nodes

```
regulator-names = "cam_vio", "cam_vana", "cam_vdig","cam_clk";
rgltr-cntrl-support;
rgltr-min-voltage = <0 3312000 1050000 0>;
rgltr-max-voltage = <0 3600000 1050000 0>;
rgltr-load-current = <0 80000 105000 0>;
```

Customers can configure different power regulators and supply using the DTSI node parameters. Also submit case to PMIC team in case of requesting different voltage.

Configure clocks

In the DTS file for each sensor node, the clock source is as follows:

```
clocks = <&clock_camcc CAM_CC_MCLK0_CLK>;
clock-names = "cam_clk";
clock-cntl-level = "turbo";
clock-rates = <24000000>;
```

The order of properties is important. The nth clock-name corresponds to the nth entry in the clock property. It is not necessary to change the list order, as it is parsed in the clock framework.

Support is provided for up to four CAM_MCLK running at 19.2 MHz by default. CAM_CLK peak-to-peak jitter < 400 ps with default frequency.

CCI master index configuration

Refer to the following table for the corresponding CCI HW mapping in DTS file CCI node

DTSI node	cci-master value	CCI HW #
&cam_cci0	0	0
&cam_cci0	1	1
&cam_cci1	0	2
&cam_cci1	1	3

If there are any shared resource such as GPIO, PIN-CTRL in between devices, please use the qcom,cam-res-mgr to configure shared resources.

Camera resource manager

If there are any shared resource such as GPIO, PIN-CTRL in between devices, use the qcom,cam-res-mgr to configure shared resources.

The following example shows sensor#0 and sensor#1 using mclk0(GPIO#94) as a shared resource. Refer to `arm64/vendor/qcom/camera/bindings/msm-cam-cci.txt`

For SM8450 and onwards refer to `vendor\qcom\proprietary\camera-devicetree\bindings\msm-cam-cci.txt`

Note: Device node should only have the `phandles` of non-shared pinctrls.

```
qcom,cam-sensor0 {
  cell-index = <0>;
  pinctrl-names = "cam_default", "cam_suspend";
  pinctrl-0 = <&cam_sensor_active_rear_wide>;
  pinctrl-1 = <&cam_sensor_suspend_rear_wide>;
  gpios = <&tlmm 94 0>, //using GPIO#94 as mclk0
  <&tlmm 93 0>;
  gpio-reset = <1>;//using the gpios[1] as reset, which is GPIO 93
gpio-req-tbl-num = <0 1>;
  gpio-req-tbl-flags = <1 0>; //0 indicates GPIO 93 as GPIOF_DIR_OUT
gpio-req-tbl-label = "CAMIF_MCLK0",
  "CAM_RESET0";
  sensor-mode = <0>;
  cci-master = <0>; status = "ok";
  clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
qcom,cam-sensor1 {
  cell-index = <1>;
  pinctrl-names = "cam_default", "cam_suspend"; pinctrl-0 = <&cam_
sensor_active_rear>; pinctrl-1 = <&cam_sensor_suspend_rear>;
```

```
  gpios = <&tlmm 94 0>, //using GPIO#94 as mclk0
  <&tlmm 95 0>;
  gpio-reset = <1>;
  gpio-req-tbl-num = <0 1>;
  gpio-req-tbl-flags = <1 0>; //0 indicates GPIO 95 as GPIOF_DIR_OUT
gpio-req-tbl-label = "CAMIF_MCLK0",
  "CAM_RESET1";
  sensor-mode = <0>;
  cci-master = <0>; status = "ok";
  clocks = <&clock_camcc CAM_CC_MCLK0_CLK>;
  clock-names = "cam_clk";
cam_res_mgr: qcom,cam-res-mgr {
  compatible = "qcom,cam-res-mgr";
  gpios-shared-pinctrl = <408>; //Assume 408 is the tlmm pin# for
GPIO#94 shared-pctrl-gpio-names = "mclk0";
  pinctrl-names = "mclk0_active", "mclk0_suspend"; pinctrl-0 = <&cam_
sensor_mclk0_active>;
  pinctrl-1 = <&cam_sensor_mclk0_suspend>; status = "ok";
};
```

Multiple shared GPIOs

If there are multiple shared GPIOs, then those shared GPIO pinctrl phandles should be defined in cam-res-mgr section only, and the phandles for the those shared GPIO pinctrl should not be defined in the device node structures which are using those shared GPIOs. For multiple shared GPIOs, define the shared gpio pinctrl in cam-res-mgr node structure and the dtsi will look like the following:

```
qcom,cam-res-mgr {
   compatible = "qcom,cam-res-mgr";
   gpios-shared-pinctrl = <408 324>; /// shared pinctrls are defined
here shared-pctrl-gpio-names = "mclk0","rst0"; //
   pinctrl-names = "mclk0_active", "mclk0_suspend","rst0_active",
"rst0_suspend";

   pinctrl-0 = <&cam_sensor_mclk0_active>;
   pinctrl-1 = <&cam_sensor_mclk0_suspend>;
   pinctrl-2 = <&cam_sensor_active_rst0>;
   pinctrl-3 = <&cam_sensor_suspend_rst0>; status = "ok";
};
```

If we assume tlmm pin #408 belongs to GPIO 100 and tlmm pin #324 belongs to GPIO 16 then the two nodes for example sensor 0 and ois0 sharing these GPIO should have the following structure:

```
ois_rear_main: qcom,ois0 {
...
   gpio-no-mux = <0>;
   //pinctrl-names = "cam_default", "cam_suspend"; // no pinctrl
phandles
   //pinctrl-0 = <&cam_sensor_mclk0_active>;
   //pinctrl-1 = <&cam_sensor_mclk0_suspend>;
   gpios = <&tlmm 100 0>,
      <&tlmm 16 0>;
   gpio-reset = <1>;
   gpio-req-tbl-num = <0 1>;
   gpio-req-tbl-flags = <1 0>;
   gpio-req-tbl-label = "CAMIF_MCLK0",
       "CAM_RESET0";
...
}

Sensor0{
   gpio-no-mux = <0>;
   //pinctrl-names = "cam_default", "cam_suspend"; //no pinctrl
phandles needed
   //pinctrl-0 = <&cam_sensor_mclk0_active>;
   //pinctrl-1 = <&cam_sensor_mclk0_suspend>;
   gpios = <&tlmm 100 0>,
       <&tlmm 16 0>;
   gpio-reset = <1>;
   gpio-req-tbl-num = <0 1>;
   gpio-req-tbl-flags = <1 0>;
   gpio-req-tbl-label = "CAMIF_MCLK0",
       "CAM_RESET0";
}
```

Ensure that the shared GPIO pinctrl is properly configured among the devices using it. If there are overlaying dtsi (OEM customized DTSI) files where a different node is using the already used/shared GPIO, ensure that the node from the overlaying dtsi file is also following the above defined configuration.

2.3 Configure sensor library

Sensor infrastructure provides an interface for customers to write their own gain/exposure update functions. Each sensor can have its own custom library, which implements the API exposed in camxsensordriverAPI.h.

After the API methods are implemented, generate the library with the name com.<vendor name>.sensor.<sensor name>.so. Include this file in the device-vendor.mk file with the other library files that need to be generated.

Generated library is available at `/vendor/etc/camera/`.

Once the library with the specified syntax name in the specified path is available, the sensor software module loads the binary and accesses the API methods to use for exposure-related functionality.

Sample library implementation is available for imx318 sensor at: `vendor/qcom/proprietary/chi-cdk/vendor/sensor/default/imx318/`

This infrastructure also allows data published using the vendor tag *org.quic.camera2.sensor_register_control* to be read and passed to the sensor library as `SensorFillExposureData -> additionalInfo`.

Library functions

It is possible to define additions to the sensor driver using a library. This library can only contain gain- specific functions with the following definitions:

```
static double RegisterToRealGain(unsigned int regGain);
static unsigned int RealToRegisterGain(double realGain);
```

Examples:

```
static double RegisterToRealGain(unsigned int regGain){ double
realGain;
    if(regGain > SENSOR_MAkX_AGAIN_REG_VAL){
    regGain = IMX318_MAX_AGAIN_REG_VAL;
    }
realGain = 512.0 / (512.0 - regGain);
return realGain;
}

static unsigned int RealToRegisterGain(double realGain){ unsigned int
regGain = 0;
    if (realGain < IMX318_MIN_AGAIN){
      realGain = SENSOR_MIN_AGAIN;
   }
  regGain = (unsigned int)(512.0 - (512.0 / realGain)); return
regGain;
}

void GetSensorLibraryAPIs(SensorLibraryAPI\* pSensorLibraryAPI){
   pSensorLibraryAPI->pRealToRegisterGain = RealToRegisterGain;
   pSensorLibraryAPI->pRegisterToRealGain = RegisterToRealGain;
```

```
}
```

2.4 Blanking requirements

The following are the IFE blanking requirements for the ISP.

For SM8150:

- Minimum horizontal blanking (pixels) – 64
- Minimum vertical blanking (lines) – 32

For SM8250 and SM8350:

- Minimum horizontal blanking (pixels) - 128
- Minimum vertical blanking (lines) - 36 for 720p input or higher

For SM8450, SM8550 and SM8650:

- Minimum horizontal blanking (pixels)
  - 128 for non HVX
  - 160 for HVX use cases
- Minimum vertical blanking (lines)
  - 90 if SFE is enabled
  - Max (MinVBI_default, MinVBI_stats) if SFE is not enabled.

  Where:
  - MinVBI _stats
    - MinVBI _stats= StatsFlushCycles/(LineTime*IFE_Freq_min) + SensLinesFlush4Stats
    - StatsFlushCycles= 8448
    - SensLinesFlush4Stats= 31 for nonHvx, 41 if HVX is enabled
  - MINVBI_default
    - MINVBI_default= 55+7us/LineTime
    - LineTime = 1/FPS/(sensorHeight + sensorVBI)

For SM8550, the following logs could provide the minimum horizonal and vertical blanking information:

```
02-12 02:54:45.582 7086 7108 I CamX : [ INFO][POWER ] camxifeutils.
cpp:1117 CalculatePixelClockRate() inputWidth=2048 SensorMinHBI=1534
SensorMinVBI=11722 IFEminHBI=64 IFEminVBI=80 SensorOpCLK 1783950000
SensorResolution=2048X1536 SFEFactor=1.000000
```

For SM8750 and SM8550:

- Minimum horizontal blanking(pixels) - 96
- Minimum vertical blanking (lines) - 64 for all usecases where input frame resolution >= 1 MP

3 Actuator bring-up guidelines

3.1 Actuator software configuration

Actuator specific XML

Every actuator has an associated configuration XML file that is used to define features such as, but not limited to, power settings, code step, and initialization settings.

Entire settings are enclosed in the < actuatorDriver></actuatorDriver> node of this XML.

Actuator information node

This node contains the information related to the actuator driver configuration parameters.

Actuator slave information

Field	Description
`slaveInfo`	Master node that stores slave information used to communicate with actuator.
`actuatorName`	Name of the actuator.
`slaveAddress`	Slave address used to talk to the actuator.
`i2cFrequencyMode`	I2C frequency mode of slave. Example: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
`actuatorType`	Actuator type. Supported types are: • VCM • BIVCM
`dataBitWidth`	Data width in bits.

`powerUpSequence`	This node contains the power-up configuration sequence required to control the power to the device while closing it. Example:
	`<powerDownSequence>`
	`  <powerSetting>`
	`    <configType>MCLK</configType>`
	`    <configValue>0</configValue>`
	`    <delayMs>0</delayMs>`
	`  </powerSetting>`
	`</powerDownSequence>`

`powerSetting`	This node contains power configuration type, value, and delay.
`configType`	Power configuration type. Example: `MCLK` Supported types are: • MCLK • VANA • VDIG • VIO • VAF • RESET • STANDBY
`configValue`	Power configuration type. Example: 24000000
`delayMs`	Delay in milliseconds. Example: 1

`powerDownSequence`	This node contains the power-down configuration sequence required to control the power to the device while closing it. Example:
	`<powerDownSequence>`
	`  <powerSetting>`
	`    <configType>RESET</configType>`
	`    <configValue>0</configValue>`
	`    <delayMs>0</delayMs>`
	`  </powerSetting>`
	`</powerDownSequence>`

Actuator register information

Field	Description
registerConfig	This node holds sequence of register configurations.
registerParam	Actuator register parameter configuration.
regAddrType	Register address type/ data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type / data size in bytes. Example: 1 * 1 = Byte data * 2 = Word data * 3 = Double word data * 4 = DT max
registerAddr	Register address that is accessed.
registerData	Register data to be programmed.
operation	Actuator operation to be performed on the register. Supported values are: • WRITE_HW_DAMP • WRITE_DAC • WRITE • WRITE_DIR_REG • POLL • READ_WRITE
delayUs	Delay in micro seconds
hwMask	Hardware mask
hwShift	Number of bits to shift for the hardware
dataShift	Number of bits to shift for data

Actuator init information

Field	Description
`initSettings`	Sequence of register settings to configure the device.
`regSetting`	Register setting configuration.
`registerAddr`	Register address that is accessed.
`registerData`	Data to be processed, read from, or written into the address.
`regAddrType`	Register address type/ data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
`regDataType`	Register data type / data size in bytes. Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
`Operation`	Operation to be performed on the register. Supported values are: • READ • WRITE • POLL
`delayUs`	Delay in micro seconds.

Actuator tuning information

Field	Description
tunedParams	Actuator tuning parameters.
initialCode	Initial DAC value.
regionParams	Actuator region parameters for all regions.
region	Actuator region parameters for single region.
macroStepBoundary	Macro step boundary in the near/forward direction.
infinityStepBoundary	Infinity step boundary in the far/backward direction.
codePerStep	Code per step.
qValue	qvalue to convert float/double to integer format.
forwardDamping	Actuator scenario ringing and damping information in forward/near direction.
backwardDamping	Actuator scenario ringing and damping information in backward/far direction.
ringingScenario	Actuator ringing scenario value.
scenarioDampingParams	Actuator damping parameters for all scenarios element for dampingStep.
scenario	Actuator damping parameters for all regions element for dampingStep.
region	Actuator damping parameters for a single region.
dampingStep	Actuator damping step.
dampingDelayUs	Actuator damping delay in micro seconds is applied after programming damping step.
hwParams	Actuator hardware parameters.

Actuator regulator

Field	Description
`compatible`	Data type that this node corresponds to.
`reg`	Points to the node ID.
`regulator-name`	LDO regulator name.
`regulator-min-microvolt`	Minimum voltage for this LDO.
`regulator-max-microvolt`	Maximum voltage for this LDO.
`regulator-enable-ramp-delay`	The time taken, in microseconds, for the supply rail to reach the target voltage, ± whatever tolerance the board design requires. This property describes the total system ramp time required due to the combination of internal ramping of the regulator itself and board design issues such as, trace capacitance and load on the supply.
`enable-active-high`	Enable with active high.
`gpio`	GPIO to be used with this node.
`vin-supply`	Input voltage supply node name.

3.2 Actuator hardware configuration

Kernel hardware configuration

Depending on the LED flash hardware, OEMs can decide which type of interface driver to configure. Some LED Flash hardware requires a power supply at input to turn it on or off. For such LED Flash hardware, OEMs can use a PMIC-based LED Flash driver to supply current or power from the PMIC IC. This driver is simple and just calls PMICAPIs to control the current or power level for different Flash states. Other LED Flash hardware is programmed with register settings to turn it on or off. For that hardware, OEMs can use either QUP or I2C-based LED Flash drivers.

The node entry in the device tree file changes based on the type of LED Flash driver – PMIC-based, I2C-based, or CCI-based.

For more details and an explanation of each field in the device tree file, refer to the kernel at `kernel/Documentation/devicetree/bindings/media/video$ vi msm-camera-flash.txt`.

Actuator node structure

Different actuator node structures are required for different interfaces. There is no support for SPI interface for actuator.

Field	Description
	SOC based actuator driver
compatible	Data type that this node corresponds to.
reg	Should contain the I2C slave address of the actuator and length of data field, which is 0x0.
regulator-name	A string used as a descriptive name for regulator outputs.
regulator-min-microvolt	Smallest voltage to set.
regulator-max-microvolt	Largest voltage to set.
regulator-enable-ramp-delay	Ramp delay for regulator.
gpio	Contains phandle to GPIO controller node and array of #gpio-cells specifying a particular GPIO.
vin-supply	phandle to a regulator that powers this actuator.

For example:

```
actuator_regulator: gpio-regulator@0 { compatible = "regulator-fixed
"; reg = <0x00 0x00>;
   regulator-name = "actuator_regulator"; regulator-min-microvolt =
<2800000>;
   regulator-max-microvolt = <2800000>;
   regulator-enable-ramp-delay = <100>; enable-active-high;
   gpio = <&tlmm 27 0>;
   vin-supply = <&pmi8998_bob>;
   };
```

Field	Description CCI-based actuator driver
`cell-index`	Contains a unique identifier to differentiate between multiple actuators.
`reg`	Corresponds to the cell-index of the node.
`compatible`	Data type to which this node corresponds to.
`qcom,cci-master`	CCI node that drives this node.
`cam_vaf-supply`	Contains the regulator from which AF voltage is supplied.
`qcom,cam-vreg-name`	Contains the names of all regulators needed by this actuator.
`qcom,cam-vreg-min-voltage`	Contains the minimum voltage level in microvolts for regulators mentioned in qcom, cam-vreg-name property (in the same order).
`qcom,cam-vreg-max-voltage`	Contains the maximum voltage level in microvolts for regulators mentioned in qcom, cam-vreg-name property (in the same order).
`qcom,cam-vreg-op-mode`	Contains the maximum current in μA that is required from the regulators mentioned in the qcom, cam-vreg-name property (in the same order).

For example:

```
actuator_front: qcom,actuator@1 { cell-index = <1>;
   reg = <0x1>;
   compatible = "qcom,actuator"; qcom,cci-master = <1>;
   cam_vaf-supply = <&actuator_regulator>; qcom,cam-vreg-name = "cam_
vaf";
   qcom,cam-vreg-min-voltage = <2800000>;
   qcom,cam-vreg-max-voltage = <2800000>;
   qcom,cam-vreg-op-mode = <0>;
   };
```

4 Flash bring-up guidelines

4.1 Flash software configuration

Flash-specific XML

Flash has an associated configuration XML file used to define features such as, but not limited to the type of flash, max current, max duration, and flash related information.

Entire settings are enclosed in the :ref:`< flashDriverData ></flashDriverData >` node of this XML.

Flash information node

Flash driver data

Field	Description
flashDriverData	This node has all the information regarding the flash type and its parameters.
flashName	Name of the flash.
flashDriverType	Flash driver type. Supported values are: • PMIC • I2C
powerUpSequence	Sequence to power up the flash.
powerDownSequence	Sequence to power down the flash.
i2cInfo	Settings for an I2C-based flash
flashInformation	Flash-related trigger information

Flash I2C information

Field	Description
`slaveAddress`	8-bit or 10-bit I2C slave write address.
`regAddrType`	Register address type/data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
`regDataType`	Register data type or data size in bytes. Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
`i2cFrequencyMode`	I2C Frequency mode of slave. Example: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
`flashInitSettings`	Sequence of I2C register settings to initialize flash.
`flashOffSettings`	Sequence of I2C register settings to turn off flash.
`flashLowSettings`	Sequence of I2C register settings to turn on low flash.
`flashHighSettings`	Sequence of I2C register settings to turn on high flash.
`regSetting`	This node holds one register configuration and forms a unit of large flash-register- settings sequence.
`registerAddr`	Register address that is accessed.
`registerData`	If the operation is WRITE, `registerData` is the data value to be written into the specified register address. If the operation is READ, `registerData` is the number of bytes to be read from the specified register address.
`regAddrType`	Type of register address.
`regDataType`	Type of register data.
`operation`	Type of operation. Supported values: • WRITE • READ • POLL
`delayUs`	Delay in microseconds. If not explicitly

Field	Description
triggerInfo	Flash trigger information.
maxFlashCurrent	Maximum flash current of the trigger in mA.
maxTorchCurrent	Maximum Torch current of the trigger in mA.
maxFlashDuration	Maximum flash duration of the trigger in ms.

Flash trigger information

4.2 Flash hardware configuration

Kernel hardware configuration

Depending on the LED flash hardware, OEMs can decide which type of interface driver to configure. Some LED flash hardware requires a power supply at input to turn it on or off. For such LED flash hardware, OEMs use a PMIC-based LED flash driver to supply current or power from the PMIC IC. This driver is simple and just calls PMIC APIs to control the current and power level for different flash states. Other LED flash hardware is programmed with register settings to turn it on/off. For that hardware, OEMs can use either QUP or I2C-based LED flash drivers.

The node entry in the device tree file changes based on the type of LED Flash driver – PMIC-based, I2C-based, or CCI-based.

The kernel at `kernel/Documentation/devicetree/bindings/media/video$ vi msm- camera-flash.txt` provides more details and explanation of each field in the device tree file.

Flash node structure

Different flash structures are required for different interfaces. There is no support for SPI interface for flash.

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
compatible	Data type that this node corresponds to.
flash-source	Should contain the array of phandles to flash source nodes.
torch-source	Should contain the phandle to Torch source node.
switch-source	Should contain the phandle to switch source node. Trigger dual led at same time to avoid sync issues.
status	Whether this node is enabled or disabled.

PMIC-based LED flash driver

For example:

```
&soc {
led_flash_rear: qcom,camera-flash@0 { cell-index = <0>;
   reg = <0x00 0x00>;
   compatible = "qcom,camera-flash";
   flash-source = <&pmi8998_flash0 &pmi8998_flash1>;
   torch-source = <&pmi8998_torch0 &pmi8998_torch1>;
   switch-source = <&pmi8998_switch0>;
   status = "ok";
};
};
```

QUP/I2C-based LED flash driver

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
qcom,slave-id	Slave address for the I2C communication with flash hardware.
compatible	Data type to which the node corresponds to.
qcom,flash-name	Flash name.
qcom,flash-type	Flash type. Supported values: • LED flash • Strobe flash • Simple LED flash controlled by one GPIO
qcom,gpio-no-mux	Specifies the GPIO mux type. • 1 = GPIO mux is not available • 0 = Otherwise
gpios	Contains phandle to GPIO controller node and array of #gpio-cells specifying a GPIO (controller-specific).
qcom,gpio-flash-en	Contains the index to GPIO used by flash enable pin of flash.
qcom,gpio-flash-now	Contains the index to GPIO used by flash's "flash now" pin.
qcom,gpio-flash-reset	Contains the index to GPIO used by flash reset pin of flash.
qcom,gpio-req-tbl-num	Contains the index to GPIO-specific to this flash.
qcom,gpio-req-tbl-flags	Contains direction of GPIO present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Contains name of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).

For example:

```
&i2c {
   led_flash0: qcom,led-flash@60 { cell-index = <0>;
   reg = <0x60>;
   qcom,slave-id = <0x60 0x00 0x0011>;
   compatible = "qcom,led-flash";
   qcom,flash-name = "adp1600";
   qcom,flash-type = <1>;
```

```
    qcom,gpio-no-mux = <0>;
    gpios = <&msmgpio 18 0>, <&msmgpio 19 0>;
    qcom,gpio-flash-en = <0>;
    qcom,gpio-flash-now = <1>;
    qcom,gpio-req-tbl-num = <0 1>;
    qcom,gpio-req-tbl-flags = <0 0>;
    qcom,gpio-req-tbl-label = "FLASH_EN", "FLASH_NOW";
   };
};
```

CCI-based LED flash driver

Field	Description
cell-index	Specifies the node ID.
reg	Increments based on the cell index.
qcom,slave-id	Slave address for the I2C communication with flash hardware.
compatible	Data type to which this node corresponds to.
label	Contain unique flash name to differentiate from other flash.
qcom,flash-type	Flash type Supported values: • LED flash • Strobe flash • Simple LED flash, controlled by one GPIO
pinctrl-names	Defined if a target uses pinctrl framework.
pinctrl-0	Pinctrl handle for pin0.
pinctrl-1	Pinctrl handle for pin1.
qcom,gpio-no-mux	Specifies the GPIO mux type. • 1 = GPIO mux is not available • 0 = Otherwise
gpios	Contain phandle to GPIO controller node and array of #gpio-cells specifying the GPIO (controller specific).
qcom,gpio-flash-en	Contains an index to GPIO used by flash enable pin of flash.
qcom,gpio-flash-now	Contains an index to GPIO used by flash now pin of flash.
qcom,gpio-flash-reset	Contains an index to GPIO used by flash reset pin of flash.
qcom,gpio-req-tbl-num	Contains an index to GPIOs specific to this flash.
qcom,gpio-req-tbl-flags	Contains the direction of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).
qcom,gpio-req-tbl-label	Contains the name of GPIOs present in qcom, gpio-req-tbl-num property (in the same order).
qcom,cci-master	Contain I2C master ID to be used for this flash. Supported values: • 0 – MASTER_0 • 1 – MASTER_1

For example:

```
$cci {
   led_flash0: qcom,led-flash@66 { cell-index = <0>;
   reg = <0x66>;
   qcom,slave-id = <0x66 0x00 0x0011>;
   compatible = "rohm-flash,bd7710";
   label = "bd7710";
   qcom,flash-type = <1>;
   qcom,gpio-no-mux = <0>; qcom,enable_pinctrl;
   pinctrl-names = "cam_flash_default", "cam_flash_suspend";
   pinctrl-0 = <&cam_sensor_flash_default>;
   pinctrl-1 = <&cam_sensor_flash_sleep>;
   gpios = <&msm_gpio 36 0>, <&msm_gpio 32 0>, <&msm_gpio 31 0>;
   qcom,gpio-flash-reset = <0>;
   qcom,gpio-flash-en = <1>;
   qcom,gpio-flash-now = <2>;
   qcom,gpio-req-tbl-num = <0 1 2>;
   qcom,gpio-req-tbl-flags = <0 0 0>;
   qcom,gpio-req-tbl-label = "FLASH_RST", "FLASH_EN", "FLASH_NOW";
  qcom,cci-master = <0>;
  };
}
```

5 EEPROM bring-up guidelines

Electrically erasable programmable read-only memory (EEPROM) is a non-volatile memory used to store the module calibration information. Module-to-module variations have an impact on the image quality of a camera system. The calibration data is used to make corrections for the module variations from a known reference (data from the golden module provided by the vendor).

Typical sources of module variation are:

- Lens placement accuracy (packaging or assembly tolerances)
- Color filter variations due to different batch in manufacturing
- IR (infrared) filter variations due to tolerance
- Electro-mechanical tolerances in autofocus actuator manufacturing
- Feature-specific calibration like PDAF, VHDR, dual camera

5.1 EEPROM software configuration

EEPROM-specific XML

EEPROM has an associated configuration XML file used to define calibration parameters such as PDAF, LSC, AF, dual camera, and WBC data.

The entire settings are enclosed in the `< EEPROMDriverData ></ EEPROMDriverData>` node of this XML.

slaveInfo node

`slaveInfo` contains the information that is used by the driver to power on/off the EEPROM device. Configuring this node is optional in kernel probe as in kernel probe EEPROM device tree node contains this information.

EEPROM slaveInfo

Field	Description
slaveInfo	Contains the sensor slave information and power settings.
EEPROMName	Name of the EEPROM. Example: atmel_at24c32e
slaveAddress	8-bit or 10-bit slave address. Example: 0xa0
regAddrType	Register address/data size in bytes. Example 2: • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register address/data size in bytes. Example 1: • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = Data type max
i2cFrequencyMode	I2C Frequency mode of slave. Example: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)

powerUpSequence	Contains the power-up configuration sequence required to control the power to the device while turning it on.
	• powerSetting: Contains power configuration type, value, and delay.
	• configType: Power configuration type. Supported types are:
	– MCLK
	– VANA
	– VDIG
	– VIO
	– VAF
	– RESET
	– STANDBY
	• configValue: Value for the specified type or 0 to use default value
	• delayMs: Delay in milliseconds.
	Example:
	`<powerUpSequence>`
	`<powerSetting>`
	`<configType>VIO</configType>`

memoryMap node

This memory map node contains the information that is needed by the driver to read the OTP data from EEPROM. Configuring this node is optional in kernel probe as in kernel probe EEPROM device tree node contains this information but at minimum one set of nodes needs to be configured containing READ operation and total size of the data in registerData. To read total size of the data for multiple sets of regSetting nodes, information will be calculated as the sum off all the registerData values corresponding to the READ operation.

EEPROM memoryMap

Field	Description
memoryMap	Contains regSetting node which contain details for read, write and poll operations. This can contain multiple sets of regSetting nodes.
regSetting	regSetting contains the slaveAddr, registerAddr, registerData, regAddrType, regDataType, operation, delayUS. Example: <regSetting> <slaveAddr>0xa0</slaveAddr> <registerAddr>0x00</registerAddr> <registerData>700</registerData> <regAddrType >2</regAddrType> <regDataType>1</regDataType> <operation>READ</operation> <delayUs>0</delayUs> </regSetting> <regSetting> <slaveAddr>0xa1</slaveAddr> <registerAddr>0x00</registerAddr> <registerData>800</registerData> <regAddrType >2</regAddrType> <regDataType>1</regDataType> <operation>READ</operation> <delayUs>0</delayUs> </regSetting>
slaveAddr	Slave address to communicate with the device. 8-bit or 10-bit slave address. Example: 0xa0
registerAddr	Register address that is accessed. Example: 0x00
registerData	Register data to be programmed/read. • For WRITE operation, registerData is the data value to be written into the specified register address. Example: 0x02

formatInfo node

The format node contains the information needed to format the OTP data obtained from EEPROM. It is not necessary to have all the sub fields filled as each EEPROM can have some or all the type of the OTP data and only those, whose data is available can be configured. In the absence of format data information, raw (unformatted) data will be published to meta data pool and that can be used to format and obtain the required data by individual modules.

For formatting the data buffer read from EEPROM following are the requirements:

- The position of each field also known as offset (with starting position reference as 0).
- Exact number of bits of data of that specific field, that is, size of the field represented in the form of mask and endianness to know the byte reading order.

In addition to the raw buffer formatting, this structure can also contain other constants that are not part of the buffer data to be configured.

EEPROM formatInfo

Field	Description
`AF`	Auto focus (AF) data information to format the AF OTP data.
`autoFocusData`	`isAvailable` – Indicates that AF data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under AF node will be ignored. Example: true `endianness` – Type of the endianness. Valid values are BIG or LITTLE Example: LITTLE
`macro`	`offset` – Position of the first byte of macro value in the buffer. Example: 22 `mask` – Represents the number of valid bits of the data read from offset Example: 0xFFF. This means 2 bytes of data to be read and in that 12 bits contain the valid data. Mask 0 indicates that particular data is not available in the buffer. The same explanation holds good for all other fields, which needs offset and mask information.

Field	Description
infinity	offset – Position of the first byte of infinity value in the buffer. mask – Represents the number of valid bits of the data.
hall	offset – Position of the first byte of hall value in the buffer. mask – Represents the number of valid bits of the data.
hallBias	offset – Position of the first byte of hall bias value in the buffer mask – Represents the number of valid bits of the data
hallRegisterAddr	Register address for hall and hall bias to be programmed. Example: 0x28
verticalMacro	offset – Position of the first byte of vertical macro value in the buffer. mask – Represents the number of valid bits of the data.
horizontalMacro	offset – Position of the first byte of horizontal macro value in the buffer. mask – Represents the number of valid bits of the data.
verticalInfinity	offset – Position of the first byte of vertical infinity value in the buffer. mask – Represents the number of valid bits of the data.
horizontalInfinity	offset – Position of the first byte of horizontal infinity value in the buffer. mask – Represents the number of valid bits of the data.
macroMargin	Margin to be extended towards macro region. Example: 0.3 that is, the new macro value is macro = macro + DACRange $\times$ 0.3 where, DACRange = macro - infinity
infinityMargin	Margin to be extended towards infinity region. Example: 0.15 that is, the new infinity value is infinity = infinity + DACRange $\times$ 0.15 where, DACRange = macro - infinity

Field	Description
WB	White balance (WB) data information to format WB data
WBData	isAvailable – Indicates that WB data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under WB node will be ignored. Example: true endianness – Type of the endianness. Valid values are BIG or LITTLE Example: LITTLE
dataType	Indicates whether the WB data is available in the form of RATIO or INDIVIDUAL. If the data is available in the form of RATIO, then rOverGValue, bOverGValue, grOverGBValue will be configured. If the data is available in the form of INDIVIDUAL, then rValue, grValue, bValue, gbValue will be configured.
lightInfo	Contains the type of the illuminant and corresponding color values based on dataType.
illuminantType	Type of the illuminant for which WB data is available. Supported illuminants are D65, TL84, A, D50, H.
rValue	offse``t – Position of the first byte of ``rValue value in the buffer. mask – Represents the number of valid bits of the data.
grValue	offset – Position of the first byte of grValue value in the buffer. mask – Represents the number of valid bits of the data.
bValue	offset – Position of the first byte of bValue value in the buffer. mask – Represents the number of valid bits of the data.
gbValue	offset – Position of the first byte of gbValue value in the buffer. mask – Represents the number of valid bits of the data.

Field	Description
rOverGValue	offset – Position of the first byte of rOverGValue value in the buffer. mask – Represents the number of valid bits of the data.
bOverGValue	offset – Position of the first byte of bOverGValue value in the buffer. mask – Represents the number of valid bits of the data.
grOverGBValue	offset – Position of the first byte of grOverGBValue value in the buffer. mask – Represents the number of valid bits of the data.
mirror	offset – Position of the first byte of mirror value in the buffer. mask – Represents the number of valid bits of the data.
flip	offset – Position of the first byte of flip value in the buffer. mask – Represents the number of valid bits of the data.
qValue	Factor to convert the obtained real values to required float values.
isInvertGROverGB	Set to true if GRoverGB needs to be inverted or else false.
LSC	Lens Shading Calibration(LSC) data needed to format LSC data.
LSCData	isAvailable – Indicates that LSC data is available or not in the OTP data buffer and valid values are true/false. If this is false, all the remaining fields under LSC node will be ignored. Example: true endianness – Type of the endianness. Valid values are BIG or LITTLE Example: LITTLE
lightInfo	Contains the type of the illuminant and corresponding LSC gain values.
illuminantType	Type of the illuminant for which LSC data is available. Supported illuminants are D65, TL84, A, D50, H.

Field	Description
rGainMSB	offset – Position of the MSB of the rGain value in the buffer. mask – Represents the number of valid bits of the data.
rGainLSB	offset – Position of the LSB of the rGain value in the buffer. mask – Represents the number of valid bits of the data.
grGainMSB	offset – Position of the MSB of the grGain value in the buffer. mask – Represents the number of valid bits of the data.
grGainLSB	offset – Position of the LSB of the grGain value in the buffer. mask – Represents the number of valid bits of the data.
gbGainMSB	offset – Position of the MSB of the gbGain value in the buffer. mask – Represents the number of valid bits of the data.
gbGainLSB	offset – Position of the LSB of the gbGain value in the buffer. mask – Represents the number of valid bits of the data.
bGainMSB	offset – Position of the MSB of the bGain value in the buffer. mask – Represents the number of valid bits of the data.
bGainLSB	offset – Position of the LSB of the bGain value in the buffer. mask – Represents the number of valid bits of the data.
meshHWRollOffSize	Mesh hardware rolloff size.

Field	Description
rIncrement	Value to be incremented to find the next rGainMSB and rGainLSB value Example: 5 If rGainMSB offset is 28, then the next rGainMSB offset will be 28+5 = 33. Same applies to other increments and if rGainLSB offset is 24, then the next rGainMSB offset will be 24+5 = 29. Same applies to other increments.
grIncrement	Value to be incremented to find next grGainMSB and grGainLSB values.
gbIncrement	Value to be incremented to find next gbGainMSB and gbGainLSB values.
bIncrement	Value to be incremented to find next bGainMSB and bGainLSB values.

5.2 EEPROM hardware configuration

Kernel hardware configuration

The kernel at kernel/Documentation/devicetree/bindings/media/video$ vi msm- camera-eeprom.txt provides more details and explanation of each field in the device tree file.

EEPROM node structure

Different EEPROM structures are required for different interfaces. SPI interface is only supported for EEPROM. The hardware configuration of the EEPROM module is saved in the kernel DTSI files.

6 PDAF bring-up guidelines

6.1 PDAF software configuration

PDAF software configuration involves:

- Updating PDAF driver XML (see PDAF-specific XML and PDAF configuration data node)
- Updating sensor driver XML (see PDAF-driver example)
- Updating module configuration

PDAF-specific XML

Every PDAF driver has an associated configuration XML file that defines features such as, but not limited to buffer format, buffer pattern, block pattern, and native format.

Entire settings are enclosed in the `< PDConfigData></ PDConfigData>` node of this XML.

PDAF configuration data node

This node contains the information related to the PDAF driver configuration parameters. This information is common to the PDAF regardless of the PDAF supported sensor resolution being picked.

Common PDAF information node

Field	Description
`PDAFName`	This is the string that contains the name of this PDAF driver.
`PDOrientation`	PD sensor orientation. Supported orientations are: • `DEFAULT` • `MIRROR` • `FLIP` • `MIRRORANDFLIP`
`PDBlackLevel`	Sensor black level information.
`PDDefocusConfidenceThreshold`	PD defocus confidence threshold.

The following node has the information about PDAF sensor native pattern to indicate the location of

PD pixels in the sensor array. Each node corresponds to one specific sensor resolution that supports PDAF nodes.

PDAF mode information node

Field	Description
PDSensorNativePatternInfo	PDAF Native Pattern for this mode.
PDBufferBlockPatternInfo	PDAF Buffer block pattern for this mode.
PDType	Specifies the PDAF type for this mode. Supported PDAF types are: • PDType2 • PDType3 • PDType2PD • PDTypeQPD
PDModeKey	This should be mapped to a valid PDAFModeKey. Each PDAF mode is given a user-defined mode key that links sensor mode to the correct PDAF mode. For example: • Sensor driver: – Sensor resolution: 0 – PDModeKey: 10 • PDAF driver: – Sensor resolution: 1 – PDModeKey: 10 In the example above, PDAF mode 1 corresponds to Sensor mode 0. The link between these modes is established using PDModeKey = 10. Max value of PDModeKey is 99.
PDAFLibraryName	Indicates the PD lib name to use.
PDSensorPDStatsFormat	Indicates the Sensor PD stats format for Type1 sensor.
PDSensorOutputFormat	Sensor Output Buffer Format for Type1 to be used for buffer allocation. Default is UNPACKED16.
lcrPDOffsetCorrection	LCR PD offset correction
PDPixelOrderType	Specifies the pixel order type. Supported order types are: • LEFTTORIGHT • RIGHTTOLEFT For 2PD sensors, this field identifies the PD pixels order when all the pixels are PD pixels.
PDOffsetCorrection	This is a floating-point value that specifies the PD Offset correction.
PDOffsetmapID (Mobile only)	PD mode for QPD sensor which has multiple pd offset maps. This should map to PDOffsetMapInfo.PDoffsetmapID read from EEPROM.

PDAF sensor native pattern node

Field	Description
PDBlockCountHorizontal	This holds the number of PD pixels in horizontal direction.
PDBlockCountVertical	This holds the number of PD pixels in vertical direction.
PDBlockPattern	PD block pattern
PDCropRegion	In-sensor cropped region
PDDownscaleFactorHorizontal	Horizontal downscale factor.
PDDownscaleFactorVertical	Vertical downscale factor.

The following node has the information about the PDAF buffer pattern that is streamed out of sensor. Each node corresponds to one specific sensor resolution that supports PDAF with maximum 14 supported nodes.

PDAF buffer block pattern node

Field	Description
PDStride	This is the number of pixels in the PD stats buffer after which there is a jump to a new line.
PDBufferFormat	This is the data type of output stats buffer. Supported values are: • MIPI8 • MIPI10 • MIPI12 • PACKED10 • UNPACKED16
PDBlockPattern	PDAF block pattern

PDAF block pattern information

Field	Description
PDPixelCount	Indicates the PDAF pixel number inside a window.
PDPixelCoordinates	Pixel 2D position, left_pixel, right_pixel. This should not contain the offsets.
PDBlockDimensions	PD Block Dimensions
PDOffsetHorizontal	Horizontal offset that should be added back for correct skip pattern.
PDOffsetVertical	Vertical offset that should be added back for correct skip pattern.

The following node has the information about PDAF pixel coordinate in the PDAF block. A maximum of 256 PDAF pixel coordinate units are allowed in a block pattern.

PDAF pixel coordinates

Field	Description
`PDXCoordinate`	This is PDAF pixel number inside a window.
`PDYCoordinate`	Pixel 2D pos, left_pixel, right_pixel. This should not contain the offsets.
`PDPixelShieldInformation`	Specifies the pixel shield information. Supported shield patterns are: • `LEFTDIODE` • `RIGHTDIODE` • `LEFTSHIELDED` • `RIGHTSHIELDED` • `UPDIODE` • `DOWNDIODE` • `UPSHIELDED` • `DOWNSHIELDED` • `TOPLEFTDIODE`

PDAF block dimensions

Field	Description
`width`	Specifies the width of the PD block dimension.
`height`	Specifies the height of the PD block dimension.

PDAF crop region information

Field	Description
`width`	Specifies the width of the PD block dimension.
`Height`	Specifies the height of the PD block dimension.
`xStart`	Specifies the starting coordinate of in-sensor crop width.
`yStart`	Specifies the starting coordinate of in-sensor crop height.

PDAF horizontal and vertical scale factors

For 2PD sensors, the vertical and horizontal downscale factors need to be calculated as follows:

In full resolution, there is no binning or scaling. Therefore, two pixels in horizontal direction will bin to one left and one right PDAF pixel. Horizontal scale factor is 2 and vertical downscale factor is 4.

For 1080p, since there is a downscale of every two pixels in sensor, both horizontal scale factor and vertical scale factor will be 4.

For HDR mode, four pixels (horizontal direction) will bin to 1 left long, 1 left short, one right long, one right short resulting in horizontal scale factor of 4 and vertical scale factor of 4.

PDAF-driver example

```
<!-- RES 1 4624x3472 @30fps -->
       <PDModeInfo>
       <PDModeKey>1</PDModeKey>
       <PDType>PDType2</PDType>

       <PDAFLibraryName>com.qti.stats.pdlib</PDAFLibraryName>
       <!--Sensor Native pattern infomation
            element for pdBlockCountHorizontal
            element for pdBlockCountVertical
            element for PD Block Pattern
            element for PD Crop Region
            element for downscale factor horizontal
            element for downscale factor vertical -->
       <PDSensorNativePatternInfo>
         <PDNativeBufferFormat>MIPI10</PDNativeBufferFormat>
         <!--Number of PD blocks in X direction 2PD: PD Image Width -
->
         <PDBlockCountHorizontal>574</PDBlockCountHorizontal>
         <!--Number of PD blocks in Y direction 2PD: PD Image Height
-->
         <PDBlockCountVertical>430</PDBlockCountVertical>
         <!--Block Pattern details of one block
             PDPixelCount: PDAF pixel number inside a window
PDPixelCoordinates: Pixel 2D pos, left_pixel,right_pixel Should not
contain the offset.
             Offset should add back for correct skip pattern. PD
Block Pattern -->
         <PDBlockPattern>
           <PDPixelCount>8</PDPixelCount>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
             <PDXCoordinate>19</PDXCoordinate>
             <PDYCoordinate>17</PDYCoordinate>
             <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
             <PDXCoordinate>20</PDXCoordinate>
             <PDYCoordinate>17</PDYCoordinate>
             <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
```

```
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
             <PDXCoordinate>17</PDXCoordinate>
             <PDYCoordinate>19</PDYCoordinate>
             <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
              <PDPixelCoordinates>
              <PDXCoordinate>18</PDXCoordinate>
              <PDYCoordinate>19</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
              <PDXCoordinate>21</PDXCoordinate>
              <PDYCoordinate>21</PDYCoordinate>
              <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
              <PDXCoordinate>22</PDXCoordinate>
              <PDYCoordinate>21</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixeCoordinates>
              <PDXCoordinate>23</PDXCoordinate>
              <PDYCoordinate>23</PDYCoordinate>
              <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
              <PDXCoordinate>24</PDXCoordinate>
              <PDYCoordinate>23</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--Width and height of the frame or subframe -->
```

```
            <PDBlockDimensions>
               <width>8</width>
               <height>8</height>
            </PDBlockDimensions>
            <PDOffsetHorizontal>17</PDOffsetHorizontal>
            <PDOffsetVertical>16</PDOffsetVertical>
         </PDBlockPattern>
            <!--Frame dimension: contains xStart, yStart, width and
height In-sensor Cropped region -->
            <PDCropRegion>
               <xStart>0</xStart>
               <yStart>0</yStart>
               <width>4624</width>
               <height>3472</height>
            </PDCropRegion>
            <!--Horizontal Downscale factor -->
            <PDDownscaleFactorHorizontal>1</
PDDownscaleFactorHorizontal>
            <!--Vertical Downscale factor -->
            <PDDownscaleFactorVertical>1</PDDownscaleFactorVertical>
         </PDSensorNativePatternInfo>
            <!--Block Pattern Info about all the blocks
                PDStride: This is the number of pixels in the PD stats
buffer after which there is a jump to a new line.
                PDBufferDataFormat: This is the data type of output
stats buffer.  -->
                <PDBufferBlockPatternInfo>
                   <PDStride>2304</PDStride>
            <!--PDAF Buffer Data Format MIPI10:       compressed, [9:
2] [9:2] [9:2] [9:2] [1:0][1:0][1:0] PACKED10: Q10 format -->
            <PDBufferFormat>UNPACKED16</PDBufferFormat>
            <!--Block Pattern details of one block
                PDPixelCount: PDAF pixel number inside a window
PDPixelCoordinates: Pixel 2D pos, left_pixel,right_pixel Should not
contain the offset.
                Offset should add back for correct skip pattern. -->
            <PDBlockPattern>
            <PDPixelCount>8</PDPixelCount>
            <!--One pixel coordinate in a block -->
            <PDPixelCoordinates>
               <PDXCoordinate>0</PDXCoordinate>
               <PDYCoordinate>0</PDYCoordinate>
               <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
```

```
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
              <PDXCoordinate>1</PDXCoordinate>
              <PDYCoordinate>0</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>

           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
           <PDXCoordinate>0</PDXCoordinate>
           <PDYCoordinate>1</PDYCoordinate>
           <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
           </PDPixelCoordinates>
           <!--One pixel coordinate in a block -->
           <PDPixelCoordinates>
              <PDXCoordinate>1</PDXCoordinate>
              <PDYCoordinate>1</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
          </PDPixelCoordinates>
          <!--One pixel coordinate in a block -->
          <PDPixelCoordinates>
              <PDXCoordinate>0</PDXCoordinate>
              <PDYCoordinate>2</PDYCoordinate>
              <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
          </PDPixelCoordinates>
          <!--One pixel coordinate in a block -->
          <PDPixelCoordinates>
             <PDXCoordinate>1</PDXCoordinate>
             <PDYCoordinate>2</PDYCoordinate>
             <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
          </PDPixelCoordinates>
          <!--One pixel coordinate in a block -->
          <PDPixelCoordinates>
              <PDXCoordinate>0</PDXCoordinate>
              <PDYCoordinate>3</PDYCoordinate>
              <PDPixelShieldInformation>RIGHTSHIELDED</
PDPixelShieldInformation>
          </PDPixelCoordinates>
```

```
          <!--One pixel coordinate in a block -->
          <PDPixelCoordinates>
              <PDXCoordinate>1</PDXCoordinate>
              <PDYCoordinate>3</PDYCoordinate>
              <PDPixelShieldInformation>LEFTSHIELDED</
PDPixelShieldInformation>
          </PDPixelCoordinates>
          <!--Width and height of the frame or subframe -->
          <PDBlockDimensions>
              <width>2</width>
              <height>4</height>
          </PDBlockDimensions>
              <PDOffsetHorizontal>0</PDOffsetHorizontal>
              <PDOffsetVertical>0</PDOffsetVertical>
          </PDBlockPattern>
          </PDBufferBlockPatternInfo>
          </PDModeInfo>
```

For QPD configuration

```
<PDModeInfo>
   <PDModeKey>0</PDModeKey>
   <PDType>PDTypeQPD</PDType>
   <PDAFLibraryName>com.qti.stats.pdlib</PDAFLibraryName>
   <PDPixelOrderType>LEFTTORIGHT</PDPixelOrderType>
   <!--Sensor Native pattern infomation element for
pdBlockCountHorizontal element for pdBlockCountVertical element for
PD Block Pattern
   element for PD Crop Region
   element for downscale factor horizontal element for downscale
factor vertical -->
   <PDSensorNativePatternInfo>
     <PDNativeBufferFormat>MIPI10</PDNativeBufferFormat>
     <!--Number of PD blocks in X direction 2PD: PD Image Width -->
     <PDBlockCountHorizontal>4096</PDBlockCountHorizontal>
     <!--Number of PD blocks in Y direction 2PD: PD Image Height -->
     <PDBlockCountVertical>3072</PDBlockCountVertical>
     <!--Frame dimension: contains xStart, yStart, width and height
In-sensor Cropped region -->
     <PDCropRegion>
       <xStart>0</xStart>
       <yStart>0</yStart>
       <width>8192</width>
       <height>6144</height>
```

```
    </PDCropRegion>
    <!--Horizontal Downscale factor -->
    <PDDownscaleFactorHorizontal>4</PDDownscaleFactorHorizontal>
    <!--Vertical Downscale factor -->
    <PDDownscaleFactorVertical>4</PDDownscaleFactorVertical>
  </PDSensorNativePatternInfo>
  <!--Block Pattern Info about all the blocks
  PDStride: This is the number of pixels in the PD stats buffer
after which there is a jump to a new line.
  PDBufferDataFormat: This is the data type of output stats buffer.
-->
  <PDBufferBlockPatternInfo>
    <PDStride>8192</PDStride>
    <!--PDAF Buffer Data Type RAW10PACKED: compressed, [9:2] [9:2]
[9:2] [9:2] [1:0][1:0][1:0][1:0] RAW10LSB: Q10 format -->
    <PDBufferFormat>UNPACKED16</PDBufferFormat>
    <PDBlockPattern>
      <PDPixelCount>4</PDPixelCount>
      <!--One pixel coordinate in a block -->
      <PDPixelCoordinates>
        <PDXCoordinate>0</PDXCoordinate>
        <PDYCoordinate>0</PDYCoordinate>
         <PDPixelShieldInformation>TOPLEFTDIODE</
PDPixelShieldInformation>
      </PDPixelCoordinates>
      <!--One pixel coordinate in a block -->
      <PDPixelCoordinates>
        <PDXCoordinate>1</PDXCoordinate>
        <PDYCoordinate>0</PDYCoordinate>
        <PDPixelShieldInformation>TOPRIGHTDIODE</
PDPixelShieldInformation>
      </PDPixelCoordinates>
      <!--One pixel coordinate in a block -->
      <PDPixelCoordinates>
        <PDXCoordinate>0</PDXCoordinate>
        <PDYCoordinate>1</PDYCoordinate>
        <PDPixelShieldInformation>BOTTOMLEFTDIODE</
PDPixelShieldInformation>
      </PDPixelCoordinates>
      <!--One pixel coordinate in a block -->
      <PDPixelCoordinates>
        <PDXCoordinate>1</PDXCoordinate>
        <PDYCoordinate>1</PDYCoordinate>
        <PDPixelShieldInformation>BOTTOMRIGHTDIODE</
```

```
PDPixelShieldInformation>
       </PDPixelCoordinates>
       <!--Width and height of the frame or subframe -->
       <PDBlockDimensions>
         <width>2</width>
         <height>2</height>
       </PDBlockDimensions>
       <PDOffsetHorizontal>0</PDOffsetHorizontal>
       <PDOffsetVertical>0</PDOffsetVertical>
     </PDBlockPattern>
   </PDBufferBlockPatternInfo>
</PDModeInfo>
```

QPD configuration

For QPD configuration, the `PDType` property value should be set to `PDTypeQPD`. Supported pixel order type could be defined in `PDPixelOrderType` attribute as `LEFTTORIGHT` or `RIGHTTOLEFT` accordingly.

Refer to `chi-cdk/oem/qcom/sensor/imx766/imx766_pdaf.xml` for QPD sample driver configuration.

For QPD mode, two conversion coefficients (one horizontal and one vertical) are expected. And up and down gain along with right and left gain map are also expected in EEPROM settings.

For more information about configuration, refer to:
`chi-cdk/oem/qcom/eeprom/gt24p128c2csli_imx766_eeprom.xml`

Sensor driver configuration for PDAF

This node contains the information related to sensor driver configuration to enable PDAF.

`resolutionInfo` node in the sensor driver XML contains all the resolutions supported by the sensor. Here, the `resolutionData` corresponding to the resolution that supports PDAF is modified to accommodate correct PDAF stream type configurations.

Also, the sensor register settings should be configured appropriately to enable PDAF data streaming based on the data sheet of the specific sensor and vendor settings.

Field	Description
`streamInfo`	This node holds all the stream configurations supported by this resolution.
`streamConfiguration`	This defines different stream configuration parameters based on the stream type.
`vc`	This is the CSID virtual channel on which we should expect the data from this stream.
`dt`	This is the data type of the data that is being streamed on this channel.
`frameDimension`	This indicates the width and height of the data being sent on the MIPI stream along with any analog cropping in the sensor, if any. If there is no crop, then xStart and yStart should be 0.
`bitWidth`	This indicates what is the MIPI output format of the stream. Example: This is 10 for MIPI10.
`type`	This indicated what is the stream type. Example: PDAF Supported values are: • BLOB • IMAGE • PDAF • HDR • META

For example:

```
<streamInfo>
  <streamConfiguration>
    <vc range="[0,3]">0</vc>
    <dt>43</dt>
    <frameDimension>
      <xStart>0</xStart>
```

```
      <yStart>0</yStart>
      <width>5488</width>
      <height>4112</height>
    </frameDimension>
    <bitWidth>10</bitWidth>
    <type>IMAGE</type>
  </streamConfiguration>
  <streamConfiguration>
    <vc range="[0,3]">0</vc>
    <dt>0x36</dt>
    <frameDimension>
      <xStart>0</xStart>
      <yStart>0</yStart>
      <width>5488</width>
      <height>2</height>
    </frameDimension>
    <bitWidth>10</bitWidth>
    <type>PDAF</type>
  </streamConfiguration>
</streamInfo>
```

Sensor XML and PDAF XML index mapping

The sensor XML is to configure the stream information with respect to each VC/DT and resolution. The PDAF XML is to configure the PD hardware block. If the sensor is outputting PDAF stream, then there should be a corresponding PD hardware configuration described in the PDAF XML. PDSensorMode is a field used in the PDAF XML for the mapping between PDAF XML and sensor XML of the resolution information corresponding to the PDAF stream. It is 1 to 1 mapped.

Field	Description
`PDSensorMode` (deprecated)	This node holds the index value of the resolution information in the sensor XML. The resolution information in the sensor XML is indexed sequentially from 0 to n-1 where 'n' represents the number of resolution data information. Deprecated for SM8450 and onwards, use `PDModeKey`.
`PDModeKey`	This should be mapped to valid PDAF Mode Key. Each P DAF Mode is given user defined mode key to link sensor mode to correct PDAF mode. For example: • Sensor driver: – ResolutionId: 2 – PDModeKey: 10 • PDAF driver: – ResolutionId: 3 – PDModeKey: 10 In the example above, PDAF mode 1 corresponds to Sensor mode 0. The link between these modes is established using `PDModeKey``equal to 10. Max value of ``PDModeKey` can be 99.

Examples of common mistakes made during adding new resolution information in sensor XML

Assume in the old resolution information there are 7 resolution information parameters: `res0`, `res1`, `res2`, `res3`, `res4`, `res5`, `res6` (where `res5` and `res6` support both IMAGE and PDAF stream).

Therefore, in PDAF XML one should see `PDSensorMode` value for 5 and 6 correspondingly. If one adds a new resolution (only IMAGE) information at index 3 of the sensor XML, there are now a total of 8 resolution information parameters: `res0`, `res1`, `res2`, `res3` (newly added), `res4` (previously `res3`), `res5` (previously `res4`), `res6` (previously `res5`), `res7` (previously `res6`).

Since the indexing for the resolution information from 3 onwards incremented by one, in the PDAF

XML the PDSensorMode value must now be changed to 6 (previously 5) and 7 (previously 6) correspondingly to align the mapping.

Original Driver Configuration				
Sensor Driver			PDAF Driver	
Sensor Mode (Auto-generated Index)	PDModeIndex (User defined key element)		PDAF Mode (Auto-generated Index)	PDModeIndex (User defined key element)
0	10		0	10
1			1	20
2	20		2	30
3	30			
4				

Add a new sensor mode in the middle of the driver				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	20
2	20		2	30
3'				
3	30			
4				

Add a new PD Mode for the newly added sensor mode				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	20
2	20		2	30
3'	40		3	40
3	30			
4				

Remove an existing sensor mode from the middle of the sensor driver				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	30
3'	40		2	40
3	30			
4				

Allows one PDAF mode to be linked to multiple sensor modes(for instance, Quadra CFA sensors with RAW stream can share same PD settings with Remosaic stream)				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	30
3'	40		2	40
3	30			
4				
5	10			

Error detection for duplicate indexing				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	30
3'	40		2	40
3	30		3	10
4				
5	10			

Error detection for an incorrect link				
Sensor Driver			PDAF Driver	
Sensor Mode	PDModeIndex		PDAF Mode	PDModeIndex
0	10		0	10
1			1	20
2	20			
3	30			
4				

7 OIS driver bring-up guidelines

7.1 OIS software configuration

OIS-specific XML

Every OIS has an associated configuration XML that defines features such as but not limited to, power settings, code step, and initialization settings.

Entire settings are enclosed in the < OISDriver></ OISDriver> node of this XML.

OIS information node

This node contains the information related to the OIS driver configuration parameters.

OIS slave information

Field	Description
slaveInfo	**Master node that stores slave information used to communicate** with the OIS.
OISName	Name of the OIS.
slaveAddress	Slave address used to talk to the OIS.
i2cFrequencyMode	I2C frequency mode of slave. Example: FAST Supported modes are: • STANDARD (100 kHz) • FAST (400 kHz) • FAST_PLUS (1 MHz) • CUSTOM (custom frequency in DTSI)
Firmware flag	Flag to know if OIS supports firmware. The following firmware coefficients need to be programmed in the xml: • Prog • Coeff • Peripheral • memory
OIS calibration flag	Flag to enable if OIS has the calibration data.

powerUpSequence	This node contains the power-up configuration sequence required to control the power to the device while closing it.
	Example:
	`<powerDownSequence>`
	`<powerSetting>`
	`<configType>MCLK</configType>`
	`<configValue>0</configValue>`
	`<delayMs>0</delayMs>`
	`</powerSetting>`
	`</ powerDownSequence>`

powerSetting	This node contains power configuration type, value, and delay.
configType	Power configuration type. Example: MCLK Supported types are: • MCLK • VANA • VDIG • VIO • VAF • RESET • STANDBY
configValue	Power configuration type. Example: 24000000

OIS register information

Field	Description
registerConfig	This node holds sequence of register configurations.
registerParam	OIS register parameter configuration.
regAddrType	Register address type/ data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type/data size in bytes Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
registerAddr	Register address that is accessed.
registerData	Register data to be programmed.
operation	OIS operation to be performed on the register. Supported values are: • WRITE_HW_DAMP • WRITE_DAC • WRITE • WRITE_DIR_REG • POLL • READ_WRITE
delayUs	Delay in micro seconds.
hwMask	Hardware mask.
hwShift	Number of bits to shift for the hardware.
dataShift	Number of bits to shift for data.

OIS init information

Field	Description
initSettings	Sequence of register settings to configure the device.
regSetting	Register setting configuration.
registerAddr	Register address that is accessed.
registerData	Data to be processed, read from, or written into the address.
regAddrType	Register address type/data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type/data size in bytes. Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
Operation	Operation to be performed on the register. Supported values are: • READ • WRITE • POLL
delayUs	Delay in micro seconds.

OIS Mode Information

Field	Description
modeSettings	OIS supports different modes pantilt mode, Movie mode, video The driver must write setting based on the mode. The settings are predefined in the xml driver and configured based on the metadata event.
regSetting	Register setting configuration.
registerAddr	Register address that is accessed.
registerData	Data to be processed, read from, or written into the address.
regAddrType	Register address type/data size in bytes. Example: 2 • 1 = Byte address • 2 = Word address • 3 = 3-byte address • 4 = Address type max
regDataType	Register data type/data size in bytes. Example: 1 • 1 = Byte data • 2 = Word data • 3 = Double word data • 4 = DT max
Operation	Operation to be performed on the register. Supported values are: • READ • WRITE • POLL
delayUs	Delay in micro seconds.

OIS regulator

Field	Description
Compatible	Data type that this node corresponds to.
`Reg`	Points to the node ID.
`regulator-name`	LDO regulator name.
`regulator-min-microvolt`	Minimum voltage for this LDO.
`regulator-max-microvolt`	Maximum voltage for this LDO.
`regulator-enable-ramp-delay`	The time taken, in microseconds, for the supply rail to reach the target voltage, ± the tolerance the board design requires. This property describes the total system ramp time required due to the combination of internal ramping of the regulator itself and board design issues such as, trace capacitance and load on the supply.
`enable-active-high`	Enable with active high.
`Gpio`	GPIO to be used with this node.
`vin-supply`	Input voltage supply node name.

OIS lens position read information

In order to read the OIS lens position data read and format information, OISLensPositionReadInfo structure needs to be updated. This is optional information and when OIS FW supports lens position read information and if that information needs to be read for EIS algorithm to use, this structure can be updated. If this information is not updated in the OIS driver xml file, it is assumed that there is no support for OIS lens position data.

This structure also supports information to update `OISLensPositionFormatInfo` and this can be updated if the raw OIS lens position data is according to the standard OIS lens position data format. If the raw data is different from the standard format, then the OIS library can be developed to implement custom function definition to format and provide the data to CAMX. Refer to KBA-200922162854 for more detail.

This is the common structure to update any of the field which needs to be formatted in `OISLensPositionFormatInfo`:

FormatInfo

Field	Description
`offset`	Indicates the position of the value to be formatted in the raw data read. Raw data index starts with 0.
`sizeInBytes`	Size of the value in bytes.
`offsetIncrementBytes`	Indicates size in bytes to increment to go to the next offset value from the current offset.

Structure definition provides information needed to format the raw data. If OEM has raw data that is not as per this standard format, they can implement OIS library function pFormatOISLensPositionData as per the API declarations in camxoisdriverapi.h. If OIS library implements this function, this will take precedence and internal formatting function will not be used. This structure is not required to update when OEMs implement OIS library format function.

OISLensPositionFormatInfo

Field	Description
`positionValidSamples`	Information to find and format number of valid lens position samples in the raw data read. Update this as per the `FormatInfo` structure details. One sample is the set of X, Y lens position data at a particular time.
`positionX`	Information to find and format X position of the OIS lens position data in the raw data read. Update this as per the `FormatInfo` structure details.
`positionY`	Information to find and format Y position of the OIS lens position data in the raw data read. Update this as per the `FormatInfo` structure details.
`timestamp`	Information to find and format timestamp of the OIS lens position data in the raw data read. Update this as per the `FormatInfo` structure details. If the timestamp is available only for the last valid lens position sample, update `offsetIncrementBytes` as 0.
`isPerSampleTimestamp`	Set it to “true” if the timestamp is available for each sample. Set it to “false” if the timestamp is available for only last valid sample
`timerClockFrequency`	Frequency of the timer clock which can be used to convert internal clock to nano seconds capacitance and load on the supply. This is optional parameter to configure and needed when timestamp unit is internal clock.
`timestampUnit`	Unit of the time stamp associated with the sample. Set it to one of the valid values from these MILLISECONDS MICROSECONDS NANOSECONDS INTERNALCLOCKTICKS

This structure defines information needed to read the OIS lens position data and write system time to OIS FW, if FW supports it and for formatting the data. If this structure is not available as part OIS driver, then framework will treat it as no OIS lens position data available.

OISLensPositionReadInfo

Field	Description
`sampleFrequencyInHZ`	Specifies the rate (in Hertz) at which OIS lens position data is captured and cached in OIS FW. This field is used to determine time difference between each lens position data and to determine OIS driver internal buffer sizes.
`totalSamples`	Specifies total number of samples in each read buffer including the dummy samples. This is a fixed value based on how many OIS samples OIS FW can cache to read.
`readIntervalTimeMS`	Specifies the interval in milli seconds at which lens position data buffer can be read. This value should be less than `((1000 / sampleFrequencyInHZ) * totalSamples)` Reduce this vale by considering software and CCI delays.
`readSettings`	Register settings to read the OIS lens position data.
`writeTimeSettings`	Element for writing reference time stamp register settings for system time synchronization. This is optional configuration and need to be configured if writing system to OIS FW is supported. Note: Write time register data should be updated to FF FF FF FF FF FF FF FF as these 8 bytes will be updated while with the current Qtimers value while writing time to OIS FW.
`formatInfo`	Format information to format the read data. Format information defined in `OISLensPositionFormatInfo`.

7.2 OIS hardware configuration

Kernel hardware configuration

The kernel at `kernel/Documentation/devicetree/bindings/media/video$ vi msm-cam- cci.txt` provides more details and explanation of each field in the device tree file.

OIS node structure

The hardware configuration of the OIS module is saved in the kernel DTSI file.

7.3 OIS library configuration

OIS infrastructure provides an interface for OEMs to write their own functions to implement custom methods. With this change, each OIS can have its own custom library, which implements the API exposed in camxoisdriverAPI.h. After the API methods are implemented, generate the library with the name com.<vendor name>.ois.<ois name>.so. This file should also be included in the device-vendor.mk file with the other library files that need to be generated.

Generated library is available at/vendor/etc/camera/.

Once the library with the specified syntax name in the specified path is available, OIS software module loads the binary and accesses the API methods to use them as needed.

Library functions

It is possible to define additional methods to the OIS driver using OIS library. This library can only expose functions with the following definitions.

Structures defined:

```
/// @brief LensPosition to hold the formated lens position data
typedef struct LensPosition
{
   FLOAT     shiftX; ///< OIS lens position shift in X direction
   FLOAT     shiftY; ///< OIS lens position shift in Y direction
   UINT64 timeStampInNS; ///< time stamp of the sample w.r.t Qtimer
in nano seconds
}LensPosition;

/// @brief LensPositionDataInfo to hold the formated lens position
and also info needs to format the raw data
typedef struct LensPositionDataInfo
{
    LensPosition*    pLensPos;       ///< OIS lens position data to
be filled in after format (Output)
    UINT     validSamples;   ///< number valid samples in the
formatted data (Output)
    UINT32   totalSamples;   ///< Total samples lensPos can hold
(Input)
    UINT8*   pRawLensPosData;        ///< Pointer to raw lens
position data (Input)
    UINT32   rawDataSize;    ///< Size of the raw lens position data
```

```
(Input)
    UINT64   QTimerInNS;     ///< QTimer values in nano seconds at
the time of lens position read (Input)
 }LensPositionDataInfo;
```

Example:

```
static bool FormatOISLensPositionData(LensPositionDataInfo*
pLensPosData)
{
// Implement the format function to format the raw data and output
the formatted data as per the define ouput structure LensPosition.
}
void GetOISLibraryAPIs(OISLibraryAPI* pOISLibraryAPI){
pOISLibraryAPI-> pFormatOISLensPositionData =
FormatOISLensPositionData;
}
```

7.4 Troubleshooting

Log analysis

Sensor reference DTSI configuration and DTSI parsing log:

```
qcom,cam-sensor4 {
cell-index = <4>; //this is to align sensor module xml compatible =
"qcom,cam-sensor"; //this should be same as
cam_sensor_driver_dt_match
csiphy-sd-index = <0>; //pair to csiphy0
actuator-src = <&actuator_triple_wide>; //including actuator
led-flash-src = <&led_flash_triple_rear>; //using LED based as Flash
eeprom-src = <&eeprom_triple_wide>; //including eeprom
cam_vio-supply = <&pm8009_l7>; cam_bob-supply = <&pm8150a_bob>; cam_
vana-supply = <&pm8009_l5>; cam_vdig-supply = <&pm8009_l1>; cam_clk-
supply = <&titan_top_gdsc>;
regulator-names = "cam_vio", "cam_vana", "cam_vdig", "cam_clk", "cam_
bob";
rgltr-cntrl-support; pwm-switch;
rgltr-min-voltage = <1800000 2800000 1104000 0 3008000>;
rgltr-max-voltage = <1800000 3000000 1104000 0 3960000>;
rgltr-load-current =<120000 80000 1200000 0 2000000>;
gpio-no-mux = <0>;
```

```
pinctrl-names = "cam_default", "cam_suspend"; //cam_default means
when running,
pinctrl-0 = <&cam_sensor_mclk0_active
&cam_sensor_active_rear>; //defined in /arm64/boot/dts/vendor/qcom/
kona- pinctrl.dtsi
pinctrl-1 = <&cam_sensor_mclk0_suspend &cam_sensor_suspend_rear>;
gpios = <&tlmm 94 0>, //using GPIO 94 as mclk0
<&tlmm 93 0>;//using GPIO 93 as reset
gpio-reset = <1>;//using the gpios[1] as reset which will pull the
pin from low to high when request, which is GPIO 93
requestgpio-req-tbl-num = <0 1>;
gpio-req-tbl-flags = <1 0>; //0 indicates GPIO 93 as GPIOF_DIR_OUT
gpio-req-tbl-label = "CAMIF_MCLK0", "CAM_RESET0";
sensor-mode = <0>;
cci-master = <0>; status = "ok";
clocks = <&clock_camcc CAM_CC_MCLK0_CLK>; clock-names = "cam_clk";
clock-cntl-level = "turbo"; clock-rates = <24000000>;
};
CAM-UTIL: cam_soc_util_get_dt_properties: 1253 cam_soc_util_get_dt_
properties for: ac4f000.qcom,cci:qcom,cam-sensor4
CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1182 cam_soc_util_get_
dt_regulator_info for: ac4f000.qcom,cci:qcom,cam-sensor4
CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1204 rgltr_name[0] =
cam_vio CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1204 rgltr_
name[1] = cam_vana
CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1204 rgltr_name[2] =
cam_vdig
CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1204 rgltr_name[3] =
cam_clk CAM-UTIL: cam_soc_util_get_dt_regulator_info: 1204 rgltr_
name[4] = cam_bob CAM-UTIL: cam_soc_util_get_dt_clk_info: 742 cam_
soc_util_get_dt_clk_info
for: ac4f000.qcom,cci:qcom,cam-sensor4
CAM-UTIL: cam_soc_util_get_dt_clk_info: 745 No shared clk parameter
defined
CAM-UTIL: cam_soc_util_get_dt_clk_info:      754     E: dev_name =
ac4f000.qcom,cci:qcom,cam-sensor4 count = 1
CAM-UTIL: cam_soc_util_get_dt_clk_info:      772     clock-names[0] =
cam_clk
CAM-UTIL: cam_soc_util_get_dt_clk_info:      820     [0] : turbo 7
CAM-UTIL: cam_soc_util_get_dt_clk_info:      840     soc_info->clk_
rate[7][0] = 24000000
CAM-UTIL: cam_soc_util_get_dt_clk_info: 848 No src_clk_str found CAM-
UTIL: cam_soc_util_get_gpio_info: 1071 gpio count 2
CAM-UTIL: cam_soc_util_get_gpio_info: 1079 gpio_array[0] = 1194
```

```
CAM-UTIL: cam_soc_util_get_gpio_info: 1079 gpio_array[1] = 1193
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1006 cam_gpio_req_tbl[0].
gpio = 1194
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1006 cam_gpio_req_tbl[1].
gpio = 1193
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1019 cam_gpio_req_tbl[0].
flags = 1
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1019 cam_gpio_req_tbl[1].
flags = 0
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1031 cam_gpio_req_tbl[0].
label = CAMIF_MCLK0
CAM-UTIL: cam_soc_util_get_dt_gpio_req_tbl: 1031 cam_gpio_req_tbl[1].
label = CAM_RESET0
CAM-UTIL: cam_soc_util_get_dt_properties: 1338 cam_soc_util_get_dt_
properties end: ac4f000.qcom,cci:qcom,cam-sensor4
```

As part of camxhal3module initialize, the sensors will be probed based on the number of sensor module BIN files in /vendor/lib64/camera and during each probe. It is based on the slot index described in module xml and DTSI.

```
CamX : [ INFO][SENSOR ] camximagesensormoduledatamanager.cpp:226
CreateAllSensorModuleSetManagers() total:8
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 0 = /vendor/lib64/camera/ com.qti.
sensormodule.truly_imx476.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 1 = /vendor/lib64/camera/ com.qti.
sensormodule.sunny_imx519.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 2 = /vendor/lib64/camera/ com.qti.
sensormodule.pmd_irs2381c.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 3 = /vendor/lib64/camera/ com.qti.
sensormodule.semco_imx481.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 4 = /vendor/lib64/camera/ com.qti.
sensormodule.ofilm_imx563.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 5 = /vendor/lib64/camera/ com.qti.
sensormodule.semco_s5k3m5.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
CreateAllSensorModuleSetManagers() 6 = /vendor/lib64/camera/ com.qti.
sensormodule.liteon_imx362.bin
CamX : [ VERB][SENSOR ] camximagesensormoduledatamanager.cpp:239
```

```
CreateAllSensorModuleSetManagers() 7 = /vendor/lib64/camera/ com.qti.
sensormodule.semco_imx586.bin
CAM_INFO: CAM-SENSOR: cam_sensor_match_id: 634 read id: 0x586
expected id 0x586:
CAM_INFO: CAM-SENSOR: cam_sensor_driver_cmd: 730 Probe success,slot:
4,slave_addr:0x34,sensor_id:0x586
CamX : [ INFO][SENSOR ] camximagesensormoduledata.cpp:736 Probe()
Probe success results - Detected: 1, DeviceIndex: 27,sensorname:
imx586
CAM_INFO: CAM-SENSOR: cam_sensor_driver_cmd: 730 Probe success,slot:
6,slave_addr:0x34,sensor_id:0x481
CamX : [ INFO][SENSOR ] camximagesensormoduledata.cpp:736 Probe()
Probe success results - Detected: 1, DeviceIndex: 28,sensorname:
imx481
CAM_INFO: CAM-SENSOR: cam_sensor_driver_cmd: 730 Probe success,slot:
5,slave_addr:0x20,sensor_id:0x30d5
CamX : [ INFO][SENSOR ] camximagesensormoduledata.cpp:736 Probe()
Probe success results - Detected: 1, DeviceIndex: 29,sensorname:
s5k3m5
semco_imx586_module.xml
<!--Module configuration -->
<moduleConfiguration description="Module configuration">
<!--CameraId is the id to which DTSI node is mapped. Typically
CameraId is the slot Id for non combo mode. -->
<cameraId>4</cameraId> kona-camera-sensor-mtp.dtsi qcom,cam-sensor4 {
cell-index = <4>;
compatible = "qcom,cam-sensor"; csiphy-sd-index = <0>;
sensor-position-roll = <90>;
```

Loading the Sensor sub modules listed in module XML

```
CamX : [ INFO][SENSOR ] camxeepromdata.cpp:104 EEPROMData() Data read
success for: cat24c64_imx586
CamX : [ INFO][SENSOR ] camximagesensormoduledata.cpp:812
CreateSensorSubModules() Actuator device found on camera 4, name:
lc898217xc
CamX : [ INFO][SENSOR ] camximagesensormoduledata.cpp:876
CreateSensorSubModules() Flash device found on camera 4.
<actuatorName>lc898217xc</actuatorName>
<eepromName>cat24c64_imx586</eepromName>
<flashName>pmic</flashName>
```

PHY data rate selection

The data rate configuration is basd on sensor stream format and output pixel clock.

```
CAM_INFO: CAM-CSIPHY: cam_csiphy_cphy_data_rate_config: 286 required
data rate : 1596333333
CAM_DBG : CAM-CSIPHY: cam_csiphy_cphy_data_rate_config: 303: table[0]
BW : 5700000000 Selected // data_rate_delta_table_1_2 in drivers
\cam_sensor_module\cam_csiphy\include\Cam_csiphy_1_2_hwreg defines
the PHY data rate and Table[0] is to support up to 2.5Gsps
```

Resolution information

The following diagram illustrate the activeDimension(green rect), frameDimension(red rect), dummyInfo(blue rect). Refer to sensor spec for more detail.

```
dummyInfo: (left, top): (right, bottom) activeDimension: (xStart1,
yStart1):Width1, Height1 frameDimension: (xStart2,yStart2):Width2,
Height2
```

PHY lane assignment (laneAssign):

Lane/trio 0 from the sensor is connected to lane/trio 0 of the MSM Rx PHY and lane/trio 1 from the sensor is connected to lane/trio 1 of the MSM Rx PHY and so on for the other lanes/trios. Below is the current lane assignment example (the change is in sensor module xml).

Sensor/TX lane#	Receiver side lane#
2	2
1	1
0	0

If OEM plans to change the lane like the following table (with some sensor register update), then the value should be laneAssign>0x021</laneAssign>.

Sensor/TX lane#	Receiver side lane#
2	0
1	2
0	1

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this “Material”), is subject to your (including the corporation or other legal entity you represent, collectively “You” or “Your”) acceptance of the terms and conditions (“Terms of Use”) set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. (“**Qualcomm Technologies**”), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as “**Qualcomm Internal Use Only**”, no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies’ prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.